

EASTER: A Six-Primitive Model for Consequential Continuity in Agent Systems

Nathan Woolen

with substantive intellectual and technical contributions from ChatGPT/Sol, Pax/Muse, Claude/Sonnet, and Hermes WitsBI

October 2026

Draft 2

Abstract

Intelligent work increasingly crosses boundaries between models, agents, sessions, processes, machines, and runtimes, yet continuity mechanisms commonly preserve conversation, application state, or runtime-specific representations rather than the structure of consequential work itself. This paper presents **EASTER** — **Evidence, Authority, State, Transition, Exception, and Receipt** — a six-primitive model for preserving consequential continuity across computational change.

We define the six primitives and implement them in a Python/SQLite reference kernel that separates consequential admission from semantic interpretation, authentication, application policy, orchestration, and model behavior. The kernel preserves immutable admitted State, explicit branching Transitions, revocable Authority, durable Evidence, Exceptions excluded from accepted State, and ACCEPTED, REJECTED, and FAILED Receipts for operations reaching the receipt-capable kernel path when outcome persistence succeeds, without imposing a canonical current state.

We evaluate EASTER in three ways. First, we test the reference implementation against its structural and authority invariants. Second, we preserve and analyze a historical six-system reverse-review as a recovered methodological record. The surviving archive retains system-level aggregate outcomes and substantial primitive-level material, but it does not preserve a complete independently reproducible classification package; the comparison is therefore not treated as a new reproducible empirical result. A complementary **Distinction-Relation-Constraint (DRC)** analysis is likewise interpreted only within that recovered coding record: all six systems were recorded as DRC-covered while some retained EASTER gaps, an observed asymmetry that motivates further testing rather than establishing a general non-entailment result. Third, we use the reference implementation during preparation of this manuscript to preserve contribution provenance across multiple human and AI participants, including corrections, kernel-admitted outcomes, and repository integration.

A dedicated closest-prior investigation finds substantial overlap between individual EASTER mechanisms and established architectural traditions; individual mechanisms are therefore not claimed as novel. Within the systems and literature examined, however, we did not identify an architecture combining live revocable permission validation on the consequential admission path, durable accepted/rejected/failed outcome distinction for attempts reaching that authoritative boundary when persistence succeeds, durable failure diagnostics excluded from accepted work-state, and deliberate refusal by the substrate to designate a canonical current semantic work-state.

These results do not establish that EASTER is minimal or universally sufficient. They motivate a narrower falsifiable hypothesis: **Evidence, Authority, State, Transition, Exception,**

and Receipt may be sufficient macroscopic variables for preserving consequential intelligent work across microscopic computational change. We identify heterogeneous-runtime continuation without access to the originating runtime’s private state as the direct experimental test of that hypothesis and define a prospective continuation oracle for that test.

1 Introduction

Intelligent systems increasingly perform work that outlives the computational context in which it was created. An agent may begin a task under one model and continue under another. A process may cross session boundaries, move between machines, survive a service restart, pass from one agent to another, or require human intervention before resuming. Long-running work may therefore persist while the model, context window, conversation history, orchestration framework, process, or runtime that produced it does not.

This creates a continuity problem.

Existing systems preserve many forms of information that help address that problem: messages, checkpoints, application state, event histories, workflow graphs, memories, databases, traces, and serialized execution context. These mechanisms are valuable, but they answer different questions. A conversation history records communication. A checkpoint may preserve execution state. A workflow graph describes possible control flow. A memory system makes selected information available later. An event log records things that occurred.

None of those categories alone specifies what must survive for **consequential work** to remain inspectable and continuable after the intelligence or runtime that produced it changes.

The distinction matters because retaining information is not the same as preserving the structure of a consequential decision.

Suppose an agent proposes a change, another participant supplies evidence, an authorized actor accepts the proposal, and a later runtime must continue from the result. Preserving only the resulting payload loses how it became admissible. Preserving only the conversation requires a later system to reconstruct the decision from narrative history. Preserving only successful operations loses rejected attempts and failures that may matter to subsequent reasoning. Treating every diagnostic event as accepted state confuses what happened during computation with what was admitted as consequential state.

A continuity substrate therefore faces several distinct questions. What supports a consequential claim or action? Who or what was permitted to act? What state was actually admitted? What change connected one admitted state to another? What failure or diagnostic condition occurred without becoming accepted state? What durable outcome records the attempted operation?

EASTER names these questions **Evidence, Authority, State, Transition, Exception, and Receipt**.

The central design move is to treat these as separate primitives rather than properties of a model, agent, conversation, workflow, or application. EASTER does not attempt to preserve the private cognitive or computational state of an intelligent system. It instead preserves a small external structure around consequential work.

This leads to a deliberately narrow architectural boundary.

EASTER does not decide what a payload means. It does not determine whether evidence is

true. It does not authenticate users, choose goals, orchestrate agents, resolve semantic disagreement, rank branches, or decide which state should be treated as the current truth. Those responsibilities remain outside the kernel. The continuity layer is concerned with whether consequential structures and their outcomes can be durably represented and validated without requiring the originating intelligence to remain present.

This separation is important for heterogeneous systems. Internal representations differ across model providers, agent frameworks, orchestration systems, applications, and runtimes. A continuity mechanism coupled to those representations inherits their boundaries. A substrate defined in terms of external consequential structure may instead permit work to cross those boundaries without requiring reconstruction of the originating system itself.

This paper investigates that possibility through a concrete architecture rather than assuming it as a property of the six primitives.

We specify EASTER as a six-primitive model and implement a reference kernel in Python and SQLite. We examine its structural behavior and use the model as a reverse-review lens across six mature agent architectures. We separately examine representational structure through a Distinction–Relation–Constraint (DRC) overlay, use the reference implementation to preserve contribution provenance during preparation of this manuscript, and investigate the closest architectural traditions against which EASTER’s combination of properties should be compared.

The evidence supports a bounded claim: EASTER is implementable as a small continuity substrate, its primitives expose meaningful distinctions when applied to existing systems, and the resulting architecture can preserve consequential provenance without owning semantic interpretation or canonical truth.

It does **not** establish that six primitives are minimal, that they are universally sufficient, or that work represented through EASTER can necessarily survive arbitrary computational change.

The stronger proposition remains a hypothesis:

Evidence, Authority, State, Transition, Exception, and Receipt may be sufficient macroscopic variables for preserving consequential intelligent work across microscopic computational change.

That proposition is useful precisely because it can fail. If a heterogeneous runtime cannot correctly continue consequential work represented through EASTER after access to the originating runtime’s private state has been removed, the failure may reveal missing primitives, inadequate projection fidelity, or continuity structure not captured by the model.

The objective is therefore not to reconstruct an intelligence.

It is to determine how little durable structure must survive for consequential work to continue without it.

2 Problem and Contributions

2.1 The continuity problem

We consider a unit of intelligent work **consequential** when its acceptance, rejection, failure, or resulting state can affect subsequent work.

The continuity problem addressed by this paper is:

What durable structure must be preserved so that consequential intelligent work remains inspectable and potentially continuable when the computational system that produced it changes or disappears?

A computational change may include replacement of a model, loss of conversational context, process restart, transfer between agents, movement between machines, replacement of an orchestration framework, or migration to a different runtime.

The problem is not equivalent to persistence.

A system can persist data while losing distinctions required to interpret the consequences of prior work. A stored payload may identify an outcome without recording who was authorized to produce it. An event history may record an attempted operation without distinguishing acceptance from failure. A checkpoint may preserve enough implementation state to restart one runtime while remaining unusable to another. A conversation may contain the evidence for a decision while requiring a later intelligence to reconstruct which statements became consequential.

For continuity across heterogeneous systems, the durable representation should therefore minimize dependence on the private implementation state of the originating runtime.

At the same time, reducing the representation too aggressively can destroy distinctions that later work requires.

EASTER investigates whether six externally representable categories provide a useful boundary between these pressures:

Evidence, Authority, State, Transition, Exception, and Receipt.

The question is not whether all computation can be represented by these primitives. Nor is it whether they reproduce the internal reasoning of the intelligence that performed the work.

The question is whether they can preserve enough of the **consequential structure** of that work for another computational context to inspect what happened and, where appropriate, continue from it.

2.2 Consequential continuity

We use **consequential continuity** to distinguish the target of EASTER from several adjacent forms of continuity.

Conversational continuity preserves prior communication or makes it recoverable. Execution continuity preserves sufficient runtime state to resume a process. Application continuity preserves domain-specific state. Memory systems preserve selected information for later retrieval. Workflow continuity preserves control structure or progress through a defined process.

These mechanisms may contribute to consequential continuity, but none is identical to it.

Consequential continuity concerns the durable relationship among what was supported, what was authorized, what became accepted state, how accepted states were connected, what consequential failures occurred outside accepted state, and what outcome was recorded for an attempted operation.

This definition deliberately does not require preservation of hidden reasoning, model activations, complete prompts, complete conversation history, or runtime-specific implementation state.

It also does not imply that a later system will interpret preserved work correctly. Preservation and interpretation are separate problems.

EASTER addresses the former.

2.3 Scope and boundary

EASTER is designed as a continuity substrate rather than an agent architecture.

The kernel therefore does not own:

- semantic interpretation of payloads;
- truth evaluation of Evidence;
- identity authentication;
- application authorization policy beyond validation of admitted Authority;
- task planning or orchestration;
- model selection or inference;
- conflict resolution among semantic claims;
- branch ranking or branch selection;
- designation of a canonical current State; or
- domain-specific rules for deciding what should happen next.

These functions belong to userland systems built around the kernel.

The distinction is intentional. If the continuity substrate must understand the semantics, policy, or private runtime representation of every system that uses it, then continuity remains coupled to those systems. EASTER instead attempts to preserve a narrower structural record that heterogeneous systems can project into and recover from.

This boundary also limits what can be inferred from successful implementation. Demonstrating that EASTER can preserve consequential structures does not demonstrate that a future agent can correctly understand them, that the original decision was correct, or that the six primitives are sufficient for every form of intelligent work.

2.4 Contributions

This paper makes five contributions.

1. **A six-primitive model of consequential continuity.** We define Evidence, Authority, State, Transition, Exception, and Receipt as separate architectural primitives and specify the distinctions among them, including immutable admitted State, explicit State-to-State Transition, revocable Authority, Exceptions outside accepted State, and durable outcomes for consequential operations.

2. **A reference implementation with a deliberately narrow kernel boundary.** We implement the model in Python and SQLite and demonstrate structural invariants while leaving semantic interpretation, authentication, application policy, orchestration, and model behavior outside the kernel. The implementation does not designate a canonical current State and permits branching rather than resolving semantic alternatives internally.
3. **A comparative reverse review across six agent architectures.** We apply the EASTER primitives as a common architectural lens to six mature systems and classify recovered support as COVERED, OPEN, GAP, or CHALLENGED. In corpus order, the recovered per-system aggregate GAP-count sequence is **(0, 0, 0, 0, 1, 3)**. This is a system-level aggregate summary, not a primitive-level vector; no preserved review artifact records a CHALLENGED disposition against an EASTER primitive. We separately apply a Distinction–Relation–Constraint overlay to test whether representational coverage entails EASTER continuity coverage in the examined corpus.
4. **An operational provenance case using EASTER itself.** During preparation of this manuscript, the reference implementation preserves contribution claims and corrections across multiple human and AI participants, including kernel-admitted operations and repository integration. This provides an in-use example of the kernel preserving consequential provenance while leaving semantic judgment and publication authority outside the kernel.
5. **A bounded architectural and experimental hypothesis.** We compare EASTER with relevant prior architectural traditions and identify a behavioral conjunction for which we did not find a matching architecture in the examined systems and literature: live revocable permission validation on the consequential admission path; durable accepted/rejected/failed outcome distinction for attempts reaching that authoritative boundary when persistence succeeds; durable failure diagnostics excluded from accepted work-state; and deliberate refusal by the substrate to designate a canonical current semantic work-state. We do not claim novelty for the individual mechanisms or exhaustive historical novelty. Instead, the resulting architecture motivates a falsifiable hypothesis: that the six EASTER primitives may constitute sufficient macroscopic variables for preserving consequential intelligent work across microscopic computational change.

2.5 Claim boundary

The paper distinguishes what has been demonstrated from what remains hypothetical.

The reference implementation demonstrates that the six distinctions can coexist in a working kernel with the specified boundary. The comparative review demonstrates how the EASTER lens classifies the examined systems under the recovered methodology. The provenance case demonstrates one operational use of the architecture.

These results provide evidence that the model is coherent and experimentally useful.

They do not establish minimality.

They do not establish universal sufficiency.

They do not establish semantic work-state portability.

The strongest claim is therefore reserved for future experimental evaluation:

If consequential work is faithfully projected through EASTER, can a heterogeneous runtime produce an acceptable next consequential action after access to the originating runtime’s private state, conversation, memory, and implementation machinery has been removed?

Section 11.6 defines the prospective oracle needed to judge such continuation without requiring Runtime B to reproduce Runtime A’s hypothetical next output. A positive result would provide evidence for semantic work-state portability. A negative result would be equally informative if it exposes a missing primitive, a projection failure, an underspecified oracle, or another continuity dependency.

The architecture is therefore presented not as a completed theory of intelligent continuity, but as a concrete substrate from which that theory can be tested.

3 EASTER: A Model for Consequential Continuity

EASTER is a six-primitive model for preserving consequential work across changes in session, agent, model, runtime, or machine. The six primitives are **Evidence, Authority, State, Transition, Exception, and Receipt**.

The model separates the continuity of consequential work from the semantic interpretation of that work. Userland determines what a payload means, whether a claim is true, which branch should be preferred, which operations are consequential enough to project to the kernel, and what consequences should follow. EASTER does not independently discover consequentiality from domain semantics. Once userland submits an operation through the EASTER boundary, the kernel defines a bounded continuity contract for that admitted projection: records are admitted under explicit Authority, state changes retain lineage, unsuccessful operations remain distinguishable from accepted State, and kernel operation outcomes remain inspectable.

The reference implementation realizes this model as a small authoritative kernel. The model and the reference implementation are distinguished throughout this paper: properties required by the EASTER continuity contract are not necessarily properties of every possible implementation, and implementation choices such as storage engine, transport, authentication mechanism, or interface are not themselves EASTER primitives.

3.1 The six primitives

These primitives are intentionally narrow. Throughout the manuscript, **Evidence, Authority, State, Transition, Exception, and Receipt** are capitalized when they refer to EASTER primitives or their corresponding kernel record types; lowercase forms retain their ordinary-language meaning. They do not attempt to reproduce an agent runtime, workflow engine, policy language, semantic model, or reasoning system inside the kernel.

3.1.1 Evidence

Evidence allows consequential work to cite durable material without asking the kernel to determine what that material means. Evidence may support later transitions or decisions, but its admission

Primitive	Role in EASTER	Boundary
Evidence	Durable, independently identifiable material that may be cited in support of consequential work.	Recording Evidence does not establish that it is true, authentic, relevant, or sufficient.
Authority	The permission represented by the EASTER Authority mechanism under which an operation may be admitted, including the lifecycle of grants and revocation.	Authority does not establish semantic correctness, truth, desirability, or application-level policy sufficiency. The reference kernel's v0.1 grant scope is narrower than a general RBAC/capability policy system.
State	Immutable admitted work-state from which later continuation may proceed.	State is not a kernel-selected current, canonical, correct, or preferred representation of the work.
Transition	The admitted lineage relation from an existing State to a newly admitted State under valid Authority.	Transition does not require global linearity; legitimate branches may share a predecessor.
Exception	Durable diagnostic material describing an operation that failed rather than becoming accepted consequential State.	An Exception is not an accepted Transition and does not itself alter authoritative State.
Receipt	Immutable kernel-authored record of the outcome of an attempted operation that reached the kernel's receipt-capable admission path.	A Receipt records the kernel outcome; it does not cover requests rejected before that boundary, guarantee persistence through storage catastrophe, semantically endorse the payload, or establish external-world causality.

establishes provenance rather than truth. A recorded observation may be mistaken; a document may be incomplete; two pieces of Evidence may conflict. Those questions remain outside the continuity contract.

This distinction allows EASTER to preserve disagreement without silently resolving it. Later work can cite the relevant Evidence while leaving interpretation to userland.

3.1.2 Authority

Authority answers whether the EASTER Authority conditions for a submitted kernel operation are satisfied at admission time. Authority is maintained independently from State: historical State does not confer future permission, and replaying or branching from an earlier State cannot restore revoked Authority.

The reference kernel supports explicit grant and revocation operations. Authority validity is evaluated as part of the consequential admission operation so that a concurrent revocation and attempted transition resolve according to serialized kernel order rather than allowing stale permission to resurrect itself through State history.

In reference kernel v0.1, grant semantics are deliberately limited. Any currently valid non-root grant may authorize an ordinary State transition when exercised by the identity that holds it; root grants additionally authorize Authority administration such as definition, grant, and revocation. The kernel does not interpret arbitrary application roles, object-level permissions, or fine-grained domain capability scope from opaque payloads. Those richer policy decisions remain userland responsibilities unless represented by mechanically enforced Authority structure.

Authority therefore establishes permission under the kernel's represented Authority rules. It does not establish that the resulting payload is correct, wise, truthful, desirable, or permitted under every external application policy.

3.1.3 State

State is an immutable admitted consequential payload. Once admitted, an existing State is not rewritten into a later condition. Change is represented by admitting another State and connecting the two through Transition.

The model permits branching. Multiple later States may legitimately descend from the same predecessor, and the kernel does not select a single branch as current, canonical, preferred, or true. Selecting which branch should guide future work remains a userland decision.

The reference kernel contains a Genesis State as the sole State permitted without an incoming Transition. Subsequent admitted States obtain lineage through Transition.

3.1.4 Transition

Transition records consequential change as an explicit relationship from one admitted State to another newly admitted State. It provides lineage without requiring the history to form one global linear chain.

Transition does not itself create Authority. An operation must already possess valid Authority to be admitted. Nor does Transition imply that a successor State is semantically better than its

predecessor; it establishes that the change was admitted under the kernel’s structural and authority rules.

3.1.5 Exception

Exception preserves diagnostic material when an operation fails rather than becoming accepted State. This separation prevents failure information from being silently confused with an admitted change to consequential work.

Exceptions are durable records when the failure-recording path itself commits successfully, but they do not become accepted State merely because they were recorded. Their role is diagnostic: they preserve what prevented an attempted operation from completing. A catastrophic failure that prevents the kernel from persisting its own failure record lies outside this durability claim.

3.1.6 Receipt

Receipt is the kernel-authored record of an operation’s outcome once an attempted operation reaches the reference kernel’s receipt-capable admission path and the outcome record itself can be persisted. The reference kernel distinguishes three ordinary post-bootstrap outcomes:

- **ACCEPTED** — the operation and its requested authoritative effects were admitted atomically.
- **REJECTED** — the kernel refused admission under its rules and the requested authoritative effects were not committed.
- **FAILED** — an exception prevented completion; the attempted authoritative effects were unwound, and a separate failure-recording transaction preserves a FAILED Receipt and diagnostic Exception when that recording transaction succeeds.

Each Receipt carries the identifier of its attempted operation. One identifier denotes one attempted operation, and the kernel admits at most one terminal Receipt per identifier (see §3.2).

Receipt therefore answers a narrower question than semantic correctness: **what durable outcome did the authoritative kernel boundary record for this attempted operation?**

This scope excludes requests refused by authentication, transport, parsing, or other components before the kernel’s receipt-capable path. It also excludes the stronger claim that a Receipt must survive a storage failure that prevents the Receipt itself from being committed. If failure-record persistence itself fails, the reference implementation rolls that recording transaction back rather than leaving a partial or phantom failure record.

Receipt ordering does not replace State/Transition lineage, and an ACCEPTED Receipt does not establish that the admitted content is true or that an external-world effect occurred exactly once.

3.2 Kernel invariants

The six primitives obtain their architectural meaning from the relationships enforced among them. The reference kernel currently embodies the following invariants.

Authority is checked on the consequential admission path. Permission is not merely advisory metadata attached after a write. An operation that requires Authority must satisfy the relevant kernel-represented validity conditions before its requested authoritative effects are admitted.

Accepted consequential change is atomic. An accepted state-changing operation commits its associated State, Transition, and ACCEPTED Receipt as one authoritative outcome. A partially admitted transition is not a successful EASTER operation.

Unsuccessful operations do not silently mutate authoritative State. REJECTED and FAILED outcomes do not commit the requested State change. After the attempted operation's transaction has unwound, the reference implementation records the unsuccessful outcome through its failure-recording path; diagnostic Exception material may be preserved without promoting that material into accepted State.

State is immutable. EASTER represents change by additional State and Transition records rather than by rewriting prior consequential history.

Lineage is explicit and branching is valid. Transition records provide State lineage. Branching is not treated as corruption, and receipt sequence or insertion order does not define a preferred branch.

The kernel does not designate current State. EASTER preserves possible consequential histories but does not decide which admitted branch should be treated as current, canonical, preferred, or semantically correct.

Authority is independent of historical State. State and Transition history cannot manufacture, restore, or imply Authority. Revocation changes future permission without rewriting previously valid history.

Evidence establishes provenance, not truth. Evidence can be durably recorded and cited without becoming a kernel-certified fact.

Exceptions remain diagnostically distinct from accepted State. Recording why an operation failed does not convert the failed operation into a successful transition.

Receipts describe kernel outcomes, not semantic endorsement. ACCEPTED means that the kernel admitted the operation according to its contract. It does not mean that the payload is true, useful, desirable, or complete.

One attempted operation has at most one terminal Receipt. An operation identifier identifies one attempted kernel operation. At most one terminal Receipt may exist for that identifier, and when durable outcome recording succeeds, exactly one exists. The reference kernel enforces this cardinality in the database schema — a unique constraint on the Receipt's operation identifier — rather than by application-level existence pre-checks, so the guarantee does not depend on entry-point discipline or race-prone read-before-write tests.

Together these invariants define the authoritative boundary more precisely than the six primitive names alone. EASTER is therefore not simply a taxonomy of records. It is a continuity model in which the primitive types have constrained relationships and distinct responsibilities.

3.3 Deliberate omissions and boundary discipline

EASTER is intentionally incomplete as an agent architecture.

The kernel does **not determine semantic truth**. It can preserve Evidence, State, lineage, Authority, failure information, and outcomes without deciding what those records mean.

The kernel does **not perform agent reasoning**. Planning, inference, model invocation, tool selection, memory retrieval, and other cognitive or orchestration functions belong outside the continuity kernel.

The kernel does **not interpret application policy**. Userland may define roles, identities, workflows, approval rules, capability structures, domain semantics, or other policy mechanisms. EASTER enforces only the authority and structural semantics mechanically represented at its boundary.

The kernel does **not decide what is consequential in the application domain**. Userland makes that semantic selection by deciding what operations and records to project through EASTER. The kernel can then preserve the continuity consequences of that projection; it cannot guarantee that userland projected every event that mattered.

The kernel does **not select a canonical branch or current State**. Branch choice is a semantic/application decision.

The model does **not claim exactly-once external execution**. A durable EASTER outcome cannot by itself prove whether an external side effect occurred once, multiple times, or outside the kernel’s observable boundary.

The reference implementation does **not claim cryptographic tamper-evidence** merely because its authoritative records are append-oriented and immutable through the supported kernel interface. Storage-level or administrative compromise is a separate security concern.

EASTER also does **not guarantee projection completeness**. The kernel can preserve only what crosses its boundary. If userland omits, collapses, or distorts consequential information before projection, EASTER may faithfully preserve an incomplete representation.

These omissions are not proposed as missing primitives. They define the scope of the continuity contract. The central design boundary is therefore:

userland owns meaning, consequentiality selection, and application policy; EASTER owns the admitted continuity record of the projection supplied to its kernel boundary.

Whether this six-primitive boundary is sufficient across heterogeneous computational change is a broader research hypothesis considered later in this paper. The architecture described here establishes the object to be tested; it does not assume the hypothesis is already true.

4 Reference Implementation

EASTER is accompanied by a reference implementation intended to make the six-primitive model executable and adversarially inspectable. The current implementation is a Python kernel over SQLite, with additional interfaces and deployment components layered outside the kernel. This section describes the implementation as evidence that the model can be instantiated; it does not treat every implementation choice as part of the EASTER model.

The implementation divides enforcement between Python and SQLite. Python owns operation semantics such as authority validation and admission behavior. SQLite supplies transactional serialization, foreign-key integrity, structural constraints, and immutability enforcement. Userland

payload meaning remains opaque to both layers except where a property must be mechanically enforced by the kernel.

4.1 Authoritative storage

The reference schema contains durable records for identities, authority definitions, authority grants and revocations, States, Evidence, Transitions, Receipts, Receipt–Evidence associations, and Exceptions.

Authoritative records are append-oriented. Database triggers reject UPDATE and DELETE operations against authoritative tables. Corrections, revocations, recovery, and later interpretation therefore occur through additional records rather than rewriting earlier history.

Payloads are represented as JSON where userland meaning is required. Structural properties that the kernel must mechanically enforce are represented separately. For example, root Authority is represented by a kernel-enforced `is\{}_root` property rather than inferred from an opaque JSON payload.

This split implements a general boundary rule of the reference kernel:

mechanically enforced kernel properties are explicit structure; application meaning remains opaque payload.

4.2 Transaction boundary

Authoritative writes execute inside SQLite `BEGIN IMMEDIATE` transactions. The kernel acquires the write transaction before performing authority validation for an operation that depends on that Authority.

This ordering is consequential. An earlier implementation allowed authority validity to be inspected before the authoritative write transaction acquired its lock. A concurrent revocation could therefore occur between validation and commit. The current implementation validates the grant using the same database connection and serialized transaction that performs the resulting write.

Accordingly, an accepted operation is evaluated against the Authority state visible at its position in the kernel’s serialized write order.

For accepted state transitions, the requested State, Transition, associated Evidence links, and ACCEPTED Receipt are committed as one transaction. If an exception escapes that transaction, the transaction context rolls back its writes rather than leaving partial admission.

The supported operation then handles the failure outside that rolled-back transaction. `record\{}_failure()` opens a new `BEGIN IMMEDIATE` transaction and writes the unsuccessful outcome as a Receipt plus linked Exception; for submitted Evidence identifiers that already exist, it may also preserve Receipt–Evidence associations. The failure-recording transaction creates no requested State or authoritative Transition. A successful failure-recording transaction therefore leaves the attempted authoritative change absent while preserving the fact and diagnostic classification of the unsuccessful attempt.

The schema additionally constrains each operation identifier to at most one terminal Receipt. A duplicate outcome record for an identifier that already has a terminal Receipt is rejected by the database itself, and the kernel surfaces the rejection as a deliberate error rather than recording a second outcome. The enforcement is therefore a schema property, not a caller-discipline convention.

This two-stage behavior is important: the FAILED Receipt and Exception are **not survivors of the transaction that failed**. The attempted operation transaction is unwound first; the failure record is a subsequent kernel transaction. If failure-record persistence itself cannot commit, that second transaction also rolls back. The implementation therefore does not claim that every catastrophic storage/process failure can produce a durable Receipt.

Regression evidence exercises this boundary directly. A genuine SQLite integrity failure and a generic unexpected exception both produce a FAILED Receipt linked to an Exception while leaving zero requested authoritative effects. Separate Exception red-team tests exercise all six supported write entry points and verify that unsuccessful operations leave no authoritative effect other than their Receipt/Exception failure record. Tests also distinguish a genuine persistence failure in `record\{}_failure()` itself: if even the fallback failure record cannot be persisted, the transaction rolls back with no phantom Receipt.

The implementation therefore relies on SQLite transaction atomicity as an implementation mechanism for two separate guarantees: accepted authoritative effects commit together with their ACCEPTED Receipt, while unsuccessful attempted effects are rolled back before any durable failure record is written.

4.3 Authority implementation

Authority is implemented independently from State and Transition history.

Authority definitions and possession are separate records. An identity possesses Authority through an immutable grant. Grants may have validity intervals and may subsequently be invalidated through append-only revocation records rather than modification of the original grant.

The current implementation supports direct revocation of a grant and `REVOKE\{}_ALL`, which invalidates grants held by an identity as of the revocation's position in Authority's ordered history.

Grant and revocation records share an Authority-owned monotonically ordered sequence. Authority validity is determined from this ledger rather than inferred from Receipt payloads or wall-clock ordering.

This separation was strengthened through adversarial remediation. A prior design derived revocation information from Receipts and used timestamps in ways that allowed clock-ordering anomalies. The current implementation makes Authority's own records the authoritative basis for Authority validity while leaving Receipts responsible for recording what the kernel did.

Root capabilities are likewise represented as mechanically enforced Authority properties. Root-authorized operations govern Authority definition, grant, and revocation, while ordinary state transition does not create or mutate Authority.

The v0.1 scope is deliberately limited. For ordinary State transitions, a currently valid non-root grant held by the requesting identity is sufficient under the kernel's Authority rules; `authority\{}_id` is not a fine-grained RBAC or object-capability policy for State content. Root grants additionally authorize Authority administration. Application-specific roles, object permissions, and semantic approval rules remain userland concerns unless they are represented by mechanically enforced kernel structure.

The result is a deliberate separation:

State history cannot manufacture Authority, and Receipt history does not deter-

mine Authority validity.

An unauthorized attempt that reaches a supported kernel operation may produce a REJECTED Receipt documenting the kernel outcome, while producing no requested authoritative mutation. Receipt records the attempt's disposition; it does not confer or substitute for Authority. Authentication or transport refusal before the kernel is reached is outside this Receipt claim.

4.4 State and Transition implementation

State rows contain immutable JSON payloads and identifiers. State does not contain its own parent pointer. Lineage is represented exclusively through Transition records connecting `from\{}_state\{}_id` to `to\{}_state\{}_id`.

The reference database begins from a distinguished Genesis State. Genesis is the sole State without an incoming Transition. Later States are admitted through successful transitions.

The schema permits multiple Transitions from the same predecessor. Repeated or competing accepted operations therefore produce separate branches rather than overwriting one another.

No schema field or kernel operation designates a canonical head or current State. Branch selection and continuation remain explicit userland choices.

Each Transition records the specific Authority grant exercised for its admission. A Transition therefore records not only lineage between States but the concrete grant under which that change entered authoritative history.

4.5 Evidence implementation

Evidence is stored as immutable, independently identifiable material.

Evidence can be associated with Receipts through a many-to-many relation. The same Evidence object may therefore support multiple operations without duplication.

The implementation deliberately distinguishes between Evidence cited in support of an operation and the Receipt created when an Evidence object itself is recorded. This prevents the provenance relation from becoming self-referential: the Receipt may establish that the kernel recorded the Evidence, but the Evidence does not thereby become support for its own creation Receipt.

Neither SQLite nor the Python kernel interprets whether Evidence is substantively true. Admission establishes that the material was durably recorded and can subsequently be cited.

4.6 Receipts and Exceptions

Receipts provide a uniform record **within the reference kernel's receipt-capable operation boundary**. The schema currently recognizes bootstrap, accepted, rejected, and failed outcomes.

An ACCEPTED Receipt may refer to an admitted Transition, but Receipt is not structurally dependent on Transition. Authority operations can succeed without producing a State Transition and still produce Receipts.

This decoupling allows Receipt to describe supported kernel operation outcomes without treating every consequential operation as a State change. It does not imply that requests refused before

reaching the kernel—for example by gateway authentication—or catastrophic failures that prevent receipt persistence themselves must have kernel Receipts.

REJECTED and FAILED operations do not produce requested authoritative Transitions. The supported entry points catch kernel-rule rejection and unexpected failure after the attempted operation transaction has unwound, then call the separate failure-recording path. `record\{}_failure()` atomically inserts the REJECTED or FAILED Receipt and its linked Exception in a new transaction. FAILED Receipts carry a minimal kernel-authored failure classification; the Exception owns the detailed diagnostics.

The failure-recording path is itself transactional. If it cannot persist a valid failure record, it rolls back rather than leaving only part of the Receipt/Exception pair. This bounds the durability claim to outcomes the kernel is able to commit; EASTER does not manufacture a durable record through an unavailable or failed persistence substrate.

The implementation therefore preserves recoverable failure without promoting failed work into authoritative State.

Receipt is also deliberately excluded from determining Authority validity. It records that the kernel performed an Authority operation; the resulting Authority records themselves determine later validity.

4.7 Genesis and bootstrap

A fresh reference database is initialized with a distinguished Genesis State and bootstrap records required to establish the initial Authority boundary.

Bootstrap is exceptional by necessity: before any Authority exists, ordinary Authority-mediated admission cannot yet authorize its own creation. The implementation therefore handles genesis explicitly rather than disguising bootstrap as an ordinary transition.

After bootstrap, consequential writes proceed through the ordinary kernel admission rules.

Genesis should therefore be understood as an implementation bootstrap boundary, not as evidence that EASTER requires one particular application-level starting state.

4.8 Interfaces are not the kernel

The repository now contains multiple components surrounding the kernel, including MCP and HTTP interfaces, consoles, a gateway, deployment configuration, and observability/read-model facilities.

These components are useful operationally, but they are not additional EASTER primitives.

The architectural requirement is that supported write interfaces project operations onto the same authoritative kernel semantics rather than independently reimplementing them. Authentication, transport security, network exposure, user interface, token distribution, and deployment policy remain boundary/userland concerns unless they affect the six-primitive continuity contract itself.

This distinction is especially important for evaluating EASTER as a model. A defect in a particular transport adapter is not automatically a defect in the six-primitive model; conversely, an interface that bypasses the authoritative kernel can violate EASTER's continuity guarantees even if the underlying database schema remains present.

4.9 Implementation scope

Several properties of the reference implementation should not be generalized into theoretical requirements without further evidence.

SQLite is an implementation choice, not an EASTER primitive.

Python is an implementation choice.

JSON is the current opaque payload representation, not a claim that consequential State must universally be encoded as JSON.

BEGIN IMMEDIATE is the mechanism used by this implementation to obtain serialized write behavior; the model requires the relevant admission invariants, not this specific database instruction.

The current root-capability model is a concrete Authority design. Other implementations may represent Authority differently provided they preserve the EASTER boundary being claimed.

Likewise, MCP, HTTP, console, gateway, and deployment mechanisms demonstrate possible access surfaces rather than defining the model.

The reference implementation therefore serves two purposes: it demonstrates that the six-primitive model can be instantiated as a working continuity kernel, and it provides a concrete artifact against which the model’s claimed invariants can be tested.

It does not establish that this implementation is the only, optimal, or universally sufficient realization of EASTER.

5 Evaluation Methodology

We evaluated EASTER through source-oriented architectural reverse review rather than through benchmark scoring. The purpose of the review was not to rank agent systems or determine whether a system was well designed. It was to ask a narrower question: within an inspectable system boundary, where are the continuity properties represented by EASTER demonstrably preserved, unresolved, absent, or challenged?

The method evolved during the research program. Draft 2 therefore distinguishes procedures that were actually used and preserved from methodological rationales reconstructed afterward. No retrospective rationale is presented as preregistered methodology.

5.1 Reverse review

For each inspected architecture, EASTER’s primitives were used as lenses against source code, public technical documentation, and other inspectable system evidence available within the review boundary.

Findings use four dispositions:

- **COVERED** — the required property is demonstrated by inspectable architecture or behavior within the reviewed boundary.
- **OPEN** — available evidence is insufficient to resolve whether the property is covered.

- **GAP** — the property is not demonstrated within the inspected system boundary.
- **CHALLENGED** — evidence attacks the adequacy of the primitive or review lens itself rather than merely showing that the inspected system does not provide it.

These labels are architectural dispositions, not scores. They are not additive measures of system quality, maturity, safety, or capability.

A GAP can represent an intentional product or architectural tradeoff. Privacy requirements, deletion guarantees, security boundaries, simplicity, cost, or limited observability may all justify not preserving a property that EASTER would preserve. Conversely, COVERED does not establish that an implementation is complete, correct, secure, or optimal.

5.1.1 Operational decision rule

The surviving research record supports a common decision rule even though the original reviews were not conducted from a single preregistered coding manual.

For each primitive-level question, the reviewer identifies the continuity property being tested, the inspectable boundary, and the strongest source or behavior available inside that boundary. The disposition is then assigned as follows:

1. **COVERED** only when the inspected evidence demonstrates the property within the stated boundary.
2. **OPEN** when the evidence does not establish either coverage or absence—for example because the relevant behavior is hidden behind a managed or compiled boundary, documentation is ambiguous, or an external-effect window cannot be resolved.
3. **GAP** when the inspected architecture exposes enough of the relevant boundary to show that the property is not provided there. Absence of public evidence alone is not sufficient for GAP when the underlying mechanism is unobservable; that case remains OPEN.
4. **CHALLENGED** when the evidence indicates that the EASTER primitive or lens itself is inadequate to describe the continuity problem, rather than merely showing a missing mechanism in the inspected system.

A disposition is therefore a claim about **evidence within a bounded architecture**, not a claim about an entire vendor or product beyond what was inspected.

This operational statement is a reconstruction of the method evidenced by the surviving reviews and their dispositions. It is included to make the interpretation of the labels reproducible. It must not be read as a claim that every historical review was executed from this exact written checklist; no such preregistered coding manual was preserved.

5.2 Primitive review and whole-system composition

The review method separates two analytical layers.

Primitive review asks whether individual EASTER properties are demonstrated at specific points in the inspected architecture.

Whole-system composition asks whether mechanisms distributed across the architecture collectively resolve, preserve, or leave open the continuity question raised at the primitive level.

This distinction became important in the recovered corpus. Hermes, OpenClaw, and Lang-Graph each retained OPEN findings during primitive-level review while producing no demonstrated aggregate GAP after whole-system composition. A primitive-level OPEN therefore cannot be mechanically promoted into a system-level GAP, nor can an aggregate zero-GAP result be interpreted as uncomplicated primitive-by-primitive coverage.

The method consequently preserves primitive findings and composition findings separately.

Whole-system composition is not an arithmetic reduction. A reviewer must identify the primitive-level concern and the additional mechanism or interaction that resolves it, leaves it OPEN, or demonstrates a GAP at the composed boundary. Where that reasoning was not preserved, Draft 2 does not reconstruct it from the final aggregate.

This limitation is particularly important for Google Antigravity. Six conceptual issue families survive archival recovery, while the frozen whole-system result contains three GAPs. The rule or case-by-case reasoning that reduced those six families to the final three was not recovered. Draft 2 therefore reports both facts and explicitly leaves the mapping unknown.

Aggregate dispositions are descriptive summaries of the frozen review outcome. They are not calculated scores and must not be reverse-engineered to invent missing primitive classifications.

5.3 Evidence discipline and unknowns

The review uses the strongest recoverable evidence available for each finding. Where source-level or primitive-level evidence has not been recovered, the classification remains unknown rather than being reconstructed from aggregate counts or researcher recollection.

This rule matters because archival completeness is uneven across the six-system corpus. Some systems retain substantial primitive-level review artifacts, while others currently retain only aggregate findings. Draft 2 therefore treats archival completeness as a separate dimension from architectural disposition.

An aggregate finding may establish that a review reached a particular disposition without establishing which primitive produced it. In such cases, the aggregate result may be reported, but the missing primitive mapping remains explicitly unknown.

Unverified recollections are not used to fill those gaps.

The September 29 recovered-review artifact [28] is itself an archival reconstruction from previously preserved research state. It records both recovered findings and explicit non-recovery. It is evidence for what the research program successfully preserved; it is not represented as the original primitive-by-primitive review package.

5.4 Review boundary

Every disposition is bounded by what could actually be inspected.

For open-source systems, this may include source code and repository documentation. For managed systems, the observable boundary may be limited to public documentation, exposed interfaces, or behavior visible to the research program.

A GAP therefore means that the property was not demonstrated within the inspected boundary **where that boundary was sufficiently observable to support an absence finding**. If available evidence cannot distinguish absence from an unobservable private mechanism, the appropriate disposition is OPEN rather than GAP.

This observability asymmetry is particularly important when comparing open and managed systems and is one reason the review is not a vendor ranking.

5.5 Corpus and chronology

The initial corpus contains six mature agent architectures. Current official documentation is provided for publication navigation, while the historical source/version limits remain those recorded in Appendix A:

- Hermes Agent [20]
- OpenClaw [21]
- LangGraph [22]
- Anthropic Claude Agent SDK [23]
- OpenAI Agents SDK [24]
- Google Antigravity [25]

The corpus is **pre-disclosure** with respect to the combined EASTER/DRC framework: these systems were not designed in response to the framework as presented in this paper.

EASTER and several EASTER reverse reviews preceded the later DRC formulation. DRC was subsequently frozen and applied to the corpus without changing its definitions to fit individual targets. Where practical, DRC classifications were frozen before overlay with prior EASTER findings.

This chronology reduces one route for retrospective fitting, but it does not create independent evaluation. The reviews were conducted within one research program involving substantial human-AI collaboration.

Accordingly, Draft 2 describes these results as **pre-disclosure architectural observations**, not independent validation or independent replication.

5.6 Corpus selection and stopping limitation

The six systems do not constitute a statistical sample.

They are mature agent architectures selected in a research program concerned with continuity, representation, memory, and agent-system behavior. The corpus is therefore biased toward systems likely to contain comparatively rich mechanisms.

The survey stopped after six systems. The rationale that repeated DRC saturation suggested additional mature frameworks would add sample count more readily than conceptual diversity was reconstructed after the stopping point rather than frozen as a preregistered stopping rule.

We therefore treat the stopping decision as a limitation, not as evidence of methodological saturation.

Untested systems remain available for later out-of-sample or independent replication rather than being retroactively incorporated into the initial corpus.

5.7 Response isolation and adversarial review

Parts of the broader research program used multiple AI collaborators for adversarial review and reconstruction.

Where reviewers were intentionally isolated, the isolation prevented direct reviewer-to-reviewer anchoring during that review round. It did not make the reviewers historically independent: they shared the same human research program and overlapping project vocabulary.

Agreement between isolated reviewers is therefore treated as convergence of independently produced responses to the same frozen material, not as independent scientific replication.

Disagreement is preserved rather than resolved by vote. Where competing interpretations affect a consequential claim, the intended resolution mechanism is evidence: inspect the underlying source or artifact, preserve the competing claims, and revise the manuscript only to the degree supported by the evidence.

5.8 Projection and interpretation limits

EASTER reverse review evaluates continuity structure, not semantic truth.

A system may preserve every reviewed continuity property while receiving an incomplete or distorted projection from userland. Conversely, a system may intentionally avoid durable preservation while maintaining rich semantic structure internally.

The method therefore does not infer semantic adequacy from EASTER coverage.

Likewise, the later DRC overlay and EASTER review operate at different layers. DRC concerns functional representational structure in userland; EASTER concerns the continuity of consequential work after projection. Their findings must not be collapsed into a single score or maturity measure.

5.9 Reproducibility status

The present corpus has uneven archival completeness. The comparative result is therefore **partially reproducible from the preserved package, not fully reproducible at primitive level for all six systems**.

For Hermes Agent, OpenClaw, and LangGraph, substantial primitive-level findings survive. For Anthropic Claude Agent SDK, only the zero-GAP aggregate survives. For OpenAI Agents SDK, the one-GAP aggregate survives but its owning primitive does not. For Google Antigravity, substantial primitive findings and six issue families survive, but the exact composition mapping to the frozen three-GAP aggregate and the Receipt freeze do not.

The publication package preserves, where available:

- exact source commit, tag, version, or review date;
- primitive-level findings;
- whole-system composition findings;
- final frozen disposition;
- source evidence supporting consequential classifications; and
- explicit unknowns where those materials were not recovered.

The September 29 recovered-review artifact [28] is the **authoritative reporting artifact for the recovered comparative state** used by the manuscript. It is pinned, together with the comparative evidence register, in the public artifact manifest associated with reference [27]. It is not the original review package, and its explicit unknowns are part of the result.

Accordingly, a reader can reproduce the derivation of the manuscript’s comparative table only to the resolution preserved by that artifact. The paper does **not** claim that a reader can independently derive every historical primitive classification or the aggregate sequence (0, 0, 0, 0, 1, 3) from a complete original review dataset, because that dataset was not preserved.

A future replication may re-review the same systems under the now-explicit decision rules, but its results would constitute a **new replication**, not a reconstruction of the missing historical cells. Such a study may confirm, narrow, or disagree with the frozen historical aggregate without changing what the original record preserved.

Missing evidence is not silently regenerated from the manuscript.

This methodology therefore supports a bounded claim: the paper reports what the preserved review process demonstrated within its inspected boundaries and identifies where that demonstration is no longer independently reproducible from the surviving archive. It does not claim exhaustive architectural knowledge, statistical representativeness, complete historical reproducibility, or independent replication.

6 Comparative Architectural Results

We applied the reverse-review method to six mature agent architectures. The results below preserve three distinct evidentiary layers: recovered primitive-level findings, recovered whole-system composition, and previously frozen aggregate disposition.

These layers are not interchangeable.

The archival record is uneven. For some systems, substantial primitive-level findings were recovered; for others, only an aggregate result survived. Missing classifications remain unknown. We do not reconstruct them from aggregate counts, present-day recollection, or later re-analysis.

The recovered-review artifact used here is itself a reconstruction created on September 29, 2026 from previously preserved research state. It is not represented as the original freeze.

6.1 Comparative summary

The table is descriptive, not ordinal. GAP counts are not system scores, and the aggregate column cannot be used to reconstruct missing primitive classifications.

In corpus order (Hermes Agent, OpenClaw, LangGraph, Anthropic Claude Agent SDK, OpenAI Agents SDK, Google Antigravity), the frozen **per-system aggregate GAP-count sequence** is **(0, 0, 0, 0, 1, 3)**. This is an ordered summary across systems, not a primitive-level vector. In particular, the surviving OpenAI aggregate does not identify which EASTER primitive owns its one GAP, and Antigravity’s recovered primitive findings do not reconstruct the exact mapping from six conceptual families to its final three-GAP aggregate.

6.2 Zero aggregate GAP did not mean complete primitive coverage

Three systems—Hermes, OpenClaw, and LangGraph—illustrate why primitive review and whole-system composition must remain separate.

Hermes retained **H-OPEN-1**, an unrecoverable external-effect window:

external effect occurs → local durable record is lost → no independent external receipt exists → later recovery cannot establish whether the effect occurred.

The finding affected Transition and Receipt but was explicitly retained as OPEN rather than promoted to GAP.

OpenClaw retained two whole-system OPEN findings.

OC-OPEN-1 has the same general external-effect shape: a non-idempotent external effect may occur, the local durable record may be lost, and no independent external receipt may exist from which later recovery can establish the event.

OC-OPEN-2 concerns delivery bypass: plugin or direct-send delivery can bypass a shared durable lifecycle, potentially leaving no authoritative later proof of delivery.

Despite these surviving OPEN findings, OpenClaw’s frozen aggregate contained no demonstrated GAP.

LangGraph likewise retained primitive-level uncertainty. Its recovered Evidence review includes an OPEN around the **UntrackedValue** boundary: whether deliberately uncheckpointed values can influence a consequential durable outcome without enough corresponding durable Evidence. Recovered Transition, Exception, and Receipt matrices also contain multiple OPEN findings.

Whole-system composition nevertheless closed many primitive-level questions through the interaction of checkpoints, pending writes, task provenance, metadata, authorization/control mechanisms, and fault-tolerance behavior.

The resulting observation is methodological rather than competitive:

zero aggregate GAP does not mean six uncomplicated COVERED primitives.

6.3 Aggregate-only results remain aggregate-only

The Anthropic Claude Agent SDK and OpenAI Agents SDK demonstrate the opposite archival problem.

System	DRC overlay	frozen	Recovered EASTER evidence	Whole-system composition	Frozen aggregate	Archival status
Hermes Agent	D/R/C ERED	COV-	Evidence, Authority, State, Exception COVERED. Transition and Receipt COVERED with surviving H-OPEN-1.	H-OPEN-1, an unrecoverable external-effect window, survives composition and remains OPEN rather than GAP.	0 demonstrated GAP; no preserved CHALLENGED disposition.	Strong primitive recovery; exact reviewed commit and individual DRC receipts not recovered.
OpenClaw	D/R/C ERED	COV-	Evidence, Authority, Exception COVERED. State and Transition COVERED + OC-OPEN-1. Receipt COVERED + OC-OPEN-1 + OC-OPEN-2. Initial sweep: 30 COVERED / 11 OPEN / 0 GAP / 0 CHALLENGED.	OC-OPEN-1 concerns an unrecoverable external-effect window. OC-OPEN-2 concerns delivery paths that may bypass a shared durable lifecycle.	0 demonstrated GAP; no preserved CHALLENGED disposition.	Strong primitive recovery; abbreviated source commit recovered; individual DRC receipts not recovered.
LangGraph	D/R/C ERED	COV-	Evidence findings substantially recovered, including an Untracked-Value OPEN. Transition: 7 COVERED / 4 OPEN. Exception: 7 COVERED / 4 OPEN. Receipt: 5 COVERED / 5 OPEN. Authority and State were reviewed but their exact matrices were not recovered.	Composition among checkpoints, tasks, pending writes, metadata, authorization/control, and fault-tolerance mechanisms closed many primitive-level OPENS.	0 demonstrated GAP; no preserved CHALLENGED disposition.	Substantial recovery; Authority/State matrices, individual DRC receipts, and exact source SHA not recovered.
Anthropic Claude Agent SDK	D/R/C ERED	COV-	Primitive-level EASTER classifications not recovered.	Not recovered.	0 demonstrated GAP.	Aggregate recovery only. Primitive decomposition remains unknown.
OpenAI Agents SDK	D/R/C ERED	COV-	Primitive-level decomposition	Not recovered.	Exactly 1 GAP.	Aggregate result and

For the Anthropic review, the frozen aggregate result of **0 GAP** survived, but the primitive-level decomposition and detailed whole-system composition did not.

We therefore report the aggregate result without inferring that all six EASTER primitives were individually COVERED.

For the OpenAI Agents SDK, the frozen aggregate establishes **exactly one real EASTER GAP**. The archival record does not establish which primitive owned that GAP.

A later recollection suggested a particular Evidence disposition, but no preserved artifact was recovered to substantiate it. That recollection is excluded from the result.

The owning primitive therefore remains unknown.

This treatment intentionally sacrifices apparent completeness in favor of provenance fidelity.

6.4 Google Antigravity

Google Antigravity retains the most substantial recovered GAP set in the initial corpus.

Recovered Evidence findings include loss of MCP annotations before policy evaluation and loss of exception fidelity, alongside COVERED call/result/step correlation and OPEN questions involving truncation and context fragmentation.

Recovered Authority findings include GAPs involving retrospective authorization provenance, Boolean human approval, dynamic delegation provenance, and silent ineffective sandbox behavior, alongside COVERED capability/authorization separation and a required authorization boundary.

Recovered State findings include COVERED recovery, trajectory/workspace separation, and concurrency protection; a GAP in compaction change metadata; and OPEN questions around exact effective context and cross-domain provenance.

Recovered Transition material includes incomplete reconstructability of compaction transitions and missing exposed provenance for chained argument transformations.

Recovered Exception findings include a GAP in structured exception fidelity, several COVERED failure behaviors, and OPEN questions involving predicate diagnostics, retry history, and durable diagnostic history.

Receipt was not sufficiently recovered and is not inferred.

At whole-system level, six conceptual families survived archival recovery:

1. evidence metadata loss;
2. retrospective authorization provenance;
3. effective-state and compaction reconstruction;
4. tool-call transformation provenance;
5. structured failure fidelity; and
6. normalized terminal outcomes.

The exact mapping from these six families to the frozen final aggregate of **three GAPs** was not recovered. We therefore do not reverse-engineer that mapping.

No preserved Antigravity review artifact records a **CHALLENGED** disposition against an **EASTER** primitive. This is an archival statement, not evidence that every primitive was affirmatively challenged and survived.

6.5 DRC/EASTER asymmetry observed in the corpus

The frozen DRC overlay classified Distinction, Relation, and Constraint as **COVERED** for all six systems.

That result should be interpreted cautiously. The corpus consists of mature agent architectures selected toward rich representation, and individual DRC primitive receipts remain incomplete for much of the corpus.

Within the recovered coding record, one bounded asymmetry was observed:

under the applied classifications in this corpus, DRC coverage co-occurred with one or more EASTER GAPs for some systems.

The OpenAI Agents SDK and Google Antigravity both received D/R/C **COVERED** dispositions while retaining one or more frozen **EASTER** GAPs within their inspected boundaries.

The converse pattern was not observed in the recovered corpus. No system in the corpus was recorded as demonstrating **EASTER** coverage while failing the DRC overlay. Given the selected mature-system corpus and incomplete primitive-level DRC receipts, this observation does not establish one-way non-entailment, a general architectural separation theorem, or whether **EASTER**-covered systems necessarily possess adequate DRC structure in userland. It is retained as an observed asymmetry that motivates prospective testing.

6.6 No preserved primitive challenge in the recovered corpus

Across the recovered six-system record, **no preserved review artifact records a CHALLENGED disposition against one of EASTER’s six primitives.**

That archival absence is not equivalent to showing that every primitive was affirmatively stress-tested in every system and survived challenge. Recovery is incomplete, and the manuscript does not infer missing primitive-level dispositions from aggregate results.

Within the preserved material, observed deficiencies could still be described using the existing **EASTER** lenses without a recovered disposition proposing a replacement or additional primitive. This does **not** establish that the six primitives are necessary, minimal, or universally sufficient.

One explicit subtraction test occurred during the LangGraph analysis. **Invariant** was considered as a possible seventh **EASTER** primitive and rejected. **Invariant** was useful for reasoning about equivalence or correctness across possible executions, but was not judged necessary for representing or reconstructing the execution that actually occurred.

This is evidence of one explicit primitive-set test, not proof of six-primitive minimality or evidence that every primitive underwent an equivalent test.

6.7 Archival result

The comparative study produced a second result about the research method itself: preserving a frozen aggregate conclusion is not equivalent to preserving a publication-ready evidence package.

Recovery quality varied substantially across systems. Some primitive findings, source pins, and composition mappings were recoverable; others were not.

The archival reconstruction therefore established a rule that Draft 2 adopts throughout:

missing archival evidence is publication work, not permission to retrospectively reclassify.

The comparative results should consequently be read as a bounded architectural record of what survived the frozen review process, including its OPEN findings and unknowns, rather than as a normalized benchmark dataset.

7 Userland and the Continuity Boundary

EASTER does not determine what consequential work means. It receives a projection of work whose functional structure has already been established outside the kernel.

This creates a boundary question:

What structure must exist in userland for consequential work to remain functionally interpretable before EASTER is asked to preserve its continuity?

During the research program, this question produced a candidate three-part vocabulary: **Distinction, Relation, and Constraint (DRC)**.

DRC and EASTER address different problems.

DRC concerns functional representational structure in userland.

EASTER concerns durable continuity of consequential work crossing the kernel boundary.

The distinction can be summarized as:

userland meaning $\rightarrow$ consequential projection $\rightarrow$ EASTER $\rightarrow$ continuity

DRC provides a vocabulary for reasoning about the structure on the userland side of that boundary. It is not proposed here as a replacement for EASTER, a lower-dimensional EASTER kernel, or a proven universal theory of meaning.

7.1 Distinction

Distinction is the ability to preserve that one consequential thing is not another.

Agent systems instantiate distinctions through such objects as messages, roles, agents, tools, memories, sessions, tasks, files, identities, checkpoints, branches, and application-specific entities.

Without adequate distinction, independently consequential entities collapse into an undifferentiated representation.

Distinction does not require one particular data model. Typed objects, identifiers, structured

records, natural-language representations, or other mechanisms may preserve distinctions when they are adequate for the work being performed.

7.2 Relation

Relation captures how distinguishable entities stand with respect to one another.

Examples include parent/child, tool-call/tool-result, agent/session, checkpoint/ancestor, Evidence/claim, task/dependency, user/instruction, and arbitrary domain relationships.

DRC does not require every Relation to be represented as an explicit typed graph edge. A natural-language representation may preserve a Relation when that representation is adequate for the consequential work.

The relevant question is whether the relationship necessary for functional interpretation remains representable, not whether a particular storage structure is used.

7.3 Constraint

Constraint describes how distinguishable and related entities may admissibly stand or evolve.

Constraints may be semantic, structural, operational, temporal, security-related, or application-defined. Examples include instructions, schemas, graph rules, tool permissions, temporal conditions, authorization requirements, and domain-specific admissibility rules.

Mutable constraints and constraints governing other constraints do not necessarily require additional DRC categories. They remain instances of Constraint so long as the representation can distinguish the relevant entities and relations and express the admissibility conditions governing their change.

In an earlier formalization, Constraint was expressed as a set of admissible successor structures:

$$\{C(G) = \{G' \mid G' \text{ is an admissible successor of } G\}$$

This notation is useful for expressing constrained evolution, but Draft 2 does not require all userland systems to instantiate a formal graph or explicitly compute such a set.

7.4 DRC as a candidate vocabulary

The claim made here is deliberately limited.

DRC proposes that **Distinction, Relation, and Constraint form a compact candidate vocabulary for asking whether a userland representation retains functional structure relevant to consequential work.**

It does not claim that every meaningful representation literally stores three fields named D, R, and C.

Nor does the present study establish that these categories are mathematically minimal, universally necessary, or sufficient for all forms of meaning.

No formal minimality proof excludes a smaller or alternative basis.

No DRC-negative architecture was observed in the initial corpus.

No ablation study established that removing one of the three categories necessarily destroys functional meaning.

The three-part reduction should therefore be treated as a research hypothesis and analytical vocabulary rather than a proven ontology.

7.5 Observed DRC coverage

The frozen DRC overlay classified all three categories as COVERED for each of the six architectures examined:

System	Distinction	Relation	Constraint
Hermes Agent	COVERED	COVERED	COVERED
OpenClaw	COVERED	COVERED	COVERED
LangGraph	COVERED	COVERED	COVERED
Anthropic Claude Agent SDK	COVERED	COVERED	COVERED
OpenAI Agents SDK	COVERED	COVERED	COVERED
Google Antigravity	COVERED	COVERED	COVERED

Thus, all three DRC categories were classified COVERED in all six examined systems.

This result is descriptive.

The systems were mature architectures selected toward comparatively rich agent functionality. Six positive cases do not establish universality, necessity, sufficiency, or strong discriminative power.

The absence of a DRC-negative specimen is consequently both an observation and a limitation.

7.6 Observed DRC/EASTER asymmetry

The same six-system corpus produced a different pattern under EASTER.

In corpus order, the frozen per-system aggregate GAP-count sequence was:

(0, 0, 0, 0, 1, 3)

This is a system-level aggregate summary, not a primitive-level vector.

OpenAI Agents SDK and Google Antigravity were classified COVERED for Distinction, Relation, and Constraint while retaining one or more EASTER GAPs within their inspected boundaries.

The recovered coding record therefore contains cases in which DRC coverage co-occurred with one or more EASTER GAPs. Because the corpus was selected toward mature architectures and individual DRC primitive receipts are incomplete, this pattern is treated as an **observed asymmetry**, not as an established non-entailment result.

The observation is consistent with the possibility that a system may preserve enough Distinction, Relation, and Constraint for functional interpretation while failing to durably preserve some consequential Evidence, Authority, State, Transition, Exception, or Receipt property across a continuity boundary. Establishing that relationship generally requires prospective testing beyond this recovered corpus.

7.7 The converse remains unobserved

The userland/continuity boundary also permits the opposite possibility in principle.

An EASTER-compatible continuity mechanism could faithfully preserve opaque consequential payloads whose userland representation is inadequate for functional interpretation.

For example, if userland collapses a distinction, loses a necessary relation, or omits a consequential constraint before projection, EASTER can preserve the resulting payload and its provenance without reconstructing the information that never crossed its boundary.

Conceptually:

EASTER-covered $\nrightarrow$ DRC-covered

However, this converse was **not observed in the present six-system corpus**.

It therefore remains an unobserved possibility under the model boundary rather than an empirical result of this study.

7.8 Projection fidelity

The boundary between userland and EASTER introduces a limitation that neither framework eliminates: **projection fidelity**.

EASTER can preserve only the consequential representation supplied to it.

If userland projects a complete and appropriately structured representation, EASTER can preserve that projection according to its continuity contract.

If userland omits consequential information, collapses distinctions, strips relations, removes constraints, or otherwise distorts the representation before admission, the kernel may faithfully preserve an incomplete artifact.

EASTER therefore provides continuity of the admitted projection, not a guarantee that the projection captures everything required for semantic reconstruction.

This is why semantic truth and application meaning remain outside the kernel.

7.9 Boundary result

The combined DRC/EASTER analysis supports a narrower observation than either a universal theory of meaning or a universal continuity theorem.

Within the recovered corpus, **DRC coverage and EASTER GAP dispositions showed an observed asymmetry under the applied classifications: some systems classified DRC-covered retained one or more EASTER GAPs**. Given the corpus selection and incomplete DRC receipts, this observation does not establish a general non-entailment relationship.

DRC provides a compact candidate vocabulary for examining functional representational structure.

EASTER provides a six-primitive model for examining and implementing consequential continuity.

Their proposed relationship is one of projection:

functional structure in userland $\rightarrow$ consequential projection $\rightarrow$ continuity boundary

Prospective testing is required to determine how strongly the observed corpus asymmetry supports that proposed division of analytical roles.

8 EASTER in Use: Contribution Provenance

The preceding sections describe EASTER as a model, an implementation, and an analytical lens. During preparation of this manuscript, the reference implementation was also used to preserve a consequential research-provenance workflow involving the human researcher and three AI participants.

This episode was not designed in advance as a controlled experiment. It is reported as an operational case study: a real provenance problem arose during preparation of the paper, Git was used to preserve the contribution artifacts, and EASTER was used to preserve consequential events surrounding their deposit and integration.

The case therefore demonstrates use of the reference implementation under its intended semantics. It does not establish universal sufficiency of the six primitives.

8.1 Provenance problem

As the manuscript developed, a publication-level attribution question arose: which contributions to EASTER and the associated research program should be attributed to Nathan Woolen, ChatGPT/Sol, Pax/Muse, and Clawde/Sonnet?

The participants had contributed in materially different ways, including architectural direction, implementation, conceptual development, prior-art investigation, adversarial review, experimental work, manuscript synthesis, and research governance.

Rather than reconstructing a single attribution ledger from one participant’s memory, the workflow preserved separate contribution accounts.

The provenance rules included:

- explicit attribution is not equivalent to historical truth;
- primary evidence may overturn corroborated recollection;
- residual or unattributed work must not automatically be assigned to another participant;
- corrections append rather than silently replacing earlier records;
- model identity and participant identity remain distinct; and
- coordination, implementation, discovery, review, authority, and acceptance remain distinguishable forms of contribution.

The purpose was therefore not to force consensus. It was to preserve attributable claims in a form that could later be compared against primary evidence.

8.2 Independent contribution accounts

The initial preserved set contained Pax/Muse’s first-party account and a provisional three-tier reconstruction of Nathan’s contributions derived from the independently produced participant accounts available at that point.

Where first-party accounts from other participants were not yet available for deposit, the archive used explicit **PENDING DEPOSIT** placeholders rather than reconstructing or fabricating their contents.

ChatGPT/Sol subsequently deposited its own first-party account. Clawde/Sonnet independently deposited its first-party account afterward.

Nathan then produced a separate first-party contribution account. His account explicitly remained **PROVISIONAL / PRE-RECONCILIATION** and stated that primary evidence should outrank recollection where the two conflict.

The resulting archive therefore preserved participant claims as claims rather than prematurely converting them into a reconciled historical truth.

8.3 Correction without erasure

The workflow encountered at least one substantive attribution correction.

An earlier contribution account attributed the architectural origin of the Authority-separation remediation too strongly to ChatGPT/Sol. Nathan subsequently clarified that the architectural decision to separate Authority from the rest of the stack and give it an independent path originated with him; ChatGPT/Sol agreed with that direction and implemented the resulting remediation, and Clawde/Sonnet later independently red-teamed it.

The correction did not require pretending that the earlier account had never existed.

The provenance convention instead allowed the original statement and its later correction to coexist, with the correction becoming the accepted account for subsequent synthesis while preserving the historical record of the mistaken attribution.

This is an example of the distinction between **historical record** and **accepted current interpretation** that EASTER’s append-oriented design is intended to support.

8.4 Git preservation

GitHub preserved the human-readable provenance artifacts and their revision history.

The first provenance-set commit, **90a1239**, created the contribution archive with the provisional reconstruction, Pax/Muse’s first-party account, pending-deposit placeholders, and an index documenting known gaps.

Later commits replaced the placeholders with first-party deposits rather than inferred reconstructions. ChatGPT/Sol’s account was deposited in commit **791b8af**; Clawde/Sonnet’s account was deposited in commit **843dec1**.

Pull request #7 then merged the multi-participant provenance set into the manuscript repository.

Nathan’s first-party account was preserved separately through pull request #8, which was sub-

sequently merged into main.

This sequence left the individual claims, their deposit chronology, and the distinction between provisional reconstruction and first-party accounts inspectable in the repository history.

8.5 EASTER preservation

EASTER was used alongside Git rather than as a replacement for it.

Git preserved the documents themselves and their repository history. EASTER preserved consequential provenance events around the workflow as attributable Evidence and Receipts.

For example, the initial contribution-record deposit was recorded as EASTER Evidence after its Git preservation. Subsequent participant deposits were likewise anchored through EASTER records, and the final merge state was recorded after both provenance pull requests had landed on main.

This produced two complementary layers:

Git → artifact content and revision history

EASTER → attributable consequential records for operations that reached the kernel boundary

The distinction is important. EASTER did not need to duplicate Git’s version-control function in order to preserve that a particular Git artifact had become consequential to the research process.

8.5.1 Preserved event-to-primitive map

The case should not be read as though every event exercised all six primitives. The surviving manuscript/repository evidence supports the following bounded map:

The final two rows state the evidence boundary directly. The case demonstrates preservation and later recovery of consequential publication history, not a self-contained proof of every external event or a canonical graph supplied by the kernel.

8.6 Merge anchoring

After the participant accounts and Nathan’s first-party account had been merged into the repository, the merge event was itself anchored in EASTER.

This extended the provenance chain from:

claim → artifact → deposit → repository integration

without requiring EASTER to determine whether every attribution claim was historically correct.

That distinction matters because the contribution accounts intentionally remained subject to later reconciliation against primary evidence.

EASTER preserved that the records were produced, deposited, and integrated. It did not certify the truth of every statement inside them.

Workflow event		Stable external identifier available in the publication repository	EASTER role demonstrated by the case	Not demonstrated by that event
Initial provenance-set preservation		Git commit 90a1239	Artifact existed as an independently identifiable provenance object; subsequent EASTER Evidence anchoring is reported for the deposit workflow.	No claim that the Git commit itself is an EASTER State or Transition.
ChatGPT/Sol party deposit	first-	Git commit 791b8af	Independently attributable contribution artifact; later anchored through the EASTER provenance workflow.	No claim that EASTER certified the historical truth of the account.
Clawde/Sonnet party deposit	first-	Git commit 843dec1	Independently attributable contribution artifact; later anchored through the EASTER provenance workflow.	No claim that every EASTER primitive was exercised by the deposit.
Multi-participant integration	inte-	Pull request #7	Repository integration point used by the provenance workflow and later merge anchoring.	Git merge is not itself an EASTER Transition unless represented as one by a kernel operation.
Nathan first-party integration	inte-	Pull request #8	Separate first-party artifact and integration point used by the later merge anchor.	No semantic reconciliation performed by the kernel.
Successful EASTER deposits / merge anchoring		Kernel operations reported by the preserved workflow	Evidence and Receipt use at the authoritative kernel boundary; accepted kernel records preserve that the relevant operation was admitted.	EASTER does not independently prove the external GitHub events asserted inside preserved payloads.
Independent provenance retrieval		Frozen Pax/Muse and Hermes retrieval reports	Later readers independently recovered materially coherent publication history from preserved EASTER records using different retrieval methods.	Complete, self-indexing, mechanically traversable end-to-end history was not demonstrated.

8.7 What the case demonstrates

The case demonstrates that the reference implementation can support a multi-participant provenance workflow in which:

- independent and provisional claims remain distinguishable;
- missing first-party evidence can remain explicitly pending rather than being fabricated;
- later corrections can supplement earlier records without requiring historical erasure;
- artifact preservation and consequential-event preservation can be separated between Git and EASTER;
- consequential integration events can be durably anchored without asking the kernel to determine semantic truth; and
- consequential portions of the preserved history can later be independently recovered from EASTER records.

These are implementation observations, not universality claims.

The episode does not establish that EASTER captures every form of research provenance, that the six primitives are sufficient for every multi-participant workflow, or that use of EASTER makes the resulting attribution historically correct. It also does not demonstrate all six primitives through every event in the workflow; the case should be read only for the records and relationships actually preserved.

It demonstrates something narrower:

the reference implementation was used to preserve consequential portions of this attribution workflow while leaving interpretation, correction, reconciliation, and publication judgment outside the kernel.

That is the boundary the implementation was designed to maintain.

8.8 Independent cold review and Receipt-cardinality remediation

After the contribution-provenance workflow above, the manuscript underwent an independent cold review by Hermes, frozen 2026-10-01T19:45:59Z against manuscript `ef81644` and reference implementation `8bf4227` (archived in `paper/reviews/hermes-independent-cold-review-2026-10-01.md`). The review produced seven substantive findings. This section concerns finding #5.

Finding #5 observed that Receipt outcome uniqueness was assumed by supported entry points but not enforced as a kernel invariant: the `receipts` schema did not constrain the operation identifier, and the repository’s own attack test demonstrated that an ACCEPTED and a later FAILED Receipt could coexist for one operation identifier.

Nathan accepted the intended invariant: **an operation identifier identifies one attempted kernel operation; at most one terminal Receipt may exist for that identifier, and when**

durable outcome recording succeeds, exactly one exists. Hermes implemented the remediation in `witsbi/easter` PR #31 — a schema-level unique constraint on the Receipt’s operation identifier, with duplicate terminal outcomes rejected as deliberate kernel errors — which was independently verified and merged as `7b8a0144dce24a2e4b3cc25a839c99ec8253c86f`. The frozen review itself was not amended: it remains byte-for-byte the assessment of the pre-remediation kernel, and a separate disposition note records finding #5 as CLOSED (`paper/reviews/hermes-independent-cold-review-fin`

The manuscript’s current implementation claims — including the Receipt-cardinality invariant in §3.2 — describe the remediated kernel. They must not be read as claims about the kernel Hermes reviewed.

EASTER anchors for this chain: Hermes’s first-party remediation record (`evidence:4be43ba1-39b7-45c1-bbe8receipt:34c03ae3-ad91-4897-ba8f-ff7efd952188`); the independent verification and merge observation (`evidence:eb26c568-751b-4630-8a7b-de20d701cbb7`).

Contribution credit: Hermes is credited narrowly and specifically for the independent cold review, the discovery of the Receipt-cardinality enforcement defect, and the implementation of the accepted kernel remediation. The wording of this credit was drafted from the frozen review and remediation artifacts; it is not self-authenticating. Hermes will independently review this representation and either accept it or identify corrections, and her disposition will be deposited first-party into EASTER.

8.9 Independent provenance retrieval

Hermes cold-review finding #3 challenged whether the provenance case exposed enough stable EASTER records to permit independent reconstruction rather than relying on a first-party narrative. The finding was tested directly rather than answered from recollection.

Pax/Muse and Hermes independently retrieved the EASTER publication history and froze separate reports before seeing the other participant’s result. Pax/Muse worked under the `agent` role using stable-ID traversal through `get\{}\_evidence` and `get\{}\_receipt`; Hermes independently enumerated the authenticated EASTER API using opaque pagination cursors and reconstructed the relevant history from the returned corpus. Their reports are preserved separately as `paper/reviews/pax-easter-provenance-retrieval-2026-10-02.md` and `paper/reviews/hermes-easter-prove` and are not treated as a synthesized canonical history.

Both retrievals recovered a materially coherent publication-provenance history from EASTER and found no conflicting consequential chronology in the records they recovered. Their coverage differed with their retrieval capabilities: Hermes’s enumeration recovered substantially more of the earlier publication history, while Pax’s restricted traversal recovered a connected later subgraph from known stable identifiers.

The two reports also converged on the principal limitation. The preserved history is not a complete, self-indexing, mechanically traversable graph. Some causal relationships are explicit stable-ID edges, while others are encoded in payload fields or require matching project, PR, commit, subject, or chronology. Later publication work is represented predominantly through Evidence rather than one continued State/Transition chain. EASTER also preserves assertions about external GitHub events without independently proving those external events.

Finding #3A is therefore closed on a bounded result: **EASTER preserved consequential publication records sufficient for two independent retrievers, using different access and**

traversal methods, to recover materially coherent publication history; complete autonomous end-to-end reconstruction was not demonstrated. This is evidence for historical preservation and recoverability, not a claim that the kernel supplies a canonical graph or graph-query semantics.

8.10 Independent clean-clone verification remediation

Hermes cold-review finding #4 identified an artifact-reproducibility weakness rather than a failure of the kernel invariants she directly tested: the repository lacked one documented, deterministic clean-clone command separating the supported verification suite from historical and development-ledger-dependent scripts. Hermes remediated that deficiency in `witsbi/easter` PR #32. After merge, review of the merged tree exposed a `PYTHONOPTIMIZE` path that could disable assertion-based checks; that intermediate deficiency was preserved rather than treated as closure, and PR #33 added fail-closed optimization handling plus a regression test for the exact condition.

Clawde/Sonnet then independently verified the merged implementation at `faacbe7e3becc7b25e37e12e5b82b578` from a fresh clone. The documented suite passed **32/32** normally and with hostile ambient `PYTHONOPTIMIZE=2`; direct optimized execution of the underlying runner exited nonzero and did not report PASS. The verifier also confirmed isolated fixture rebuilding, inclusion of the Receipt-cardinality and optimization regressions, explicit separation of historical/non-suite material, and no changes by PRs #32/#33 to `kernel.py`, `schema.sql`, or the six-primitive semantics.

The remediation and its correction were preserved as separate EASTER records rather than collapsed into a single successful narrative. Clawde’s independent paper-side verification is `evidence:a788ed8c-5b34-4`. Nathan subsequently accepted finding #4 as CLOSED, with that decision recorded by Pax/Muse as an explicit proxy record at `evidence:906988cd-27df-4161-a8a9-0fd3c7088bbd`. This closure strengthens artifact reproducibility; it does not add a new EASTER primitive or change the kernel semantics described in §4.

9 Related Work and Novelty Boundary

EASTER intersects several established architectural traditions. Individual EASTER properties appear in event-sourced systems, durable-execution engines, distributed ledgers, capability and authorization systems, provenance frameworks, version-control systems, and agent runtimes.

The claim of this paper is therefore not that immutable history, authorization, receipts, branching, provenance, or durable failure recording are individually new.

The relevant question is narrower:

Do existing architectures examined in this research combine the particular continuity properties EASTER places together at one consequential admission boundary?

We conducted a dedicated closest-prior investigation to test that question rather than treating absence from the initial agent-system corpus as evidence of novelty.

9.1 Search method and evidentiary warrant

The closest-prior work was a **bounded adversarial investigation, not a systematic or exhaustive literature review**. This characterization is supported by two independently deposited first-party methodology accounts and the contemporaneous research artifacts.

Two AI-assisted passes approached the question with different decompositions and different search shapes.

Pax/Muse pass. Pax/Muse designed a six-property rubric corresponding to the EASTER combination, an explicit kill condition, and a bounded research brief, then delegated web investigation to an isolated deep-research agent. The brief pre-identified Hyperledger Fabric as the lead candidate and named adjacent candidates including Ethereum, QLDB, Temporal/Cadence, Kafka, Git, CRDT stores, Certificate Transparency, TUF, Axon, and provenance/lineage systems. The research agent inspected documentation, specifications, repositories, RFCs, and related sources and returned a report that Pax/Muse reviewed. Pax/Muse did not personally re-open the twelve cited sources during that pass. The resulting investigation is therefore best described as a directed kill-attempt against a pre-named strong contender, not as an open-ended discovery survey.

Clawde/Sonnet pass. Clawde/Sonnet separately used a deep-research workflow for a broader prior-art investigation: six parallel research subagents examined pre-selected traditions including event sourcing/ledgers, durable execution/fault recovery, provenance, capability security, ER/type systems, and ontology/knowledge representation, followed by a synthesis agent. A later closest-architecture pass then examined Fabric and, reactively after Fabric’s canonicity weakness became salient, Corda and Holochain. That later pass used an eight-property decomposition whose pre-use derivation artifact was not recovered. Whether the later Fabric/Corda/Holochain dispatch itself formally used the named deep-research skill is also unrecovered; its delegation pattern was similar, but the record does not justify treating that as confirmed.

The two passes were independent of one another in the relevant sense: their analyses were produced without either participant first harmonizing its result to the other’s. They were **not independent of AI research tooling**, and most source reading occurred in delegated research contexts rather than being personally re-fetched by the orchestrating participant.

The passes were subsequently reconciled against their cited source claims. The preserved reconciliation records no material contradiction in the underlying primary-source facts; the principal difference was decomposition and weighting. The four-property behavioral conjunction reported below emerged from that reconciliation rather than from either pass alone.

This procedure supports a narrow warrant: strong pre-identified and adjacent candidate architectures were subjected to differently decomposed attempts to defeat the conjunction, and none of the examined systems did so. It does **not** support the claim that the literature contains no closer architecture or that the candidate space was exhausted.

9.1.1 Source discipline

The investigations preferred primary technical sources where accessible: official project documentation and repositories, specifications and RFCs, project or vendor whitepapers, and academic papers. Preserved source records include, among others, Hyperledger Fabric documentation and source plus the EuroSys Fabric paper; the Corda technical whitepaper; the Holochain whitepaper and developer documentation; W3C PROV material; RFC 6749; primary capability-security literature; Temporal,

Kafka, Axon, EventStoreDB, and Git documentation; and a formal CRDT treatment.

Primary-source access was not complete. The Pax/Muse pass used a third-party mirror for the Corda whitepaper. Clawde/Sonnet’s pass records inaccessible or unparsed primary material in several areas and labels secondary or search-cache substitution rather than silently presenting it as direct primary inspection. Neither first-party account documents a primary-vs-primary factual conflict requiring adjudication.

The final submission bibliography should cite the authoritative original source for each manuscript claim where an equivalent accessible original can be verified. Replacing a historical mirror with an authoritative bibliographic citation improves the publication apparatus; it does not rewrite which source was actually inspected during the historical pass.

9.1.2 Search limitations preserved rather than repaired retrospectively

The historical search has known holes. Pax/Muse’s brief named TUF, OpenLineage, Marquez, DataHub, and Atlas, but the resulting report does not show whether those candidates were examined and rejected or never examined. Clawde/Sonnet’s reconstruction identifies an unexamined class of permissioned distributed architectures combining hard revocable admission with non-canonical or sharded replication. Candidate selection was not governed by formal inclusion/exclusion criteria, database queries, citation chaining, or a PRISMA-like screening protocol. The work was time-boxed to essentially one evening.

These gaps are not filled retrospectively. A later search of them would constitute additional prior-art work, not recovery of the historical procedure.

9.2 Architectural neighborhoods

Several neighboring traditions overlap with EASTER while solving different primary problems.

Event sourcing and durable execution preserve histories from which application state can be reconstructed and can durably represent failures or retries. Their primary abstraction is generally execution or domain-event continuity rather than an independent six-part admission record separating Evidence, Authority, State, Transition, Exception, and Receipt.

Distributed ledgers provide particularly strong neighboring examples because they combine immutable histories, validation rules, identity or endorsement mechanisms, and transaction outcomes. Hyperledger Fabric, Corda, Ethereum, and Holochain therefore received dedicated comparison.

Version-control and append-only structures such as Git, Certificate Transparency, CRDT-oriented stores, and related systems demonstrate that branching or refusal of a single application-level canonical state can coexist with durable history. These systems generally do not also provide the behavioral combination examined below: revocable permission checked on the admission path, durable outcomes for accepted/rejected/failed attempts that reach that authoritative path, durable failure diagnostics excluded from accepted work-state, and no substrate-designated canonical current work-state.

Provenance and supply-chain systems, including in-toto-like designs, strongly represent evidence and attestation but do not necessarily own the admission and continuity semantics EASTER places inside the kernel.

Agent frameworks remain the most direct application neighborhood because EASTER arose from continuity problems observed in intelligent work. Section 6 reports those systems separately rather than treating them as the complete prior-art universe.

These overlaps motivate a conjunction claim rather than a primitive-by-primitive novelty claim.

9.3 Hyperledger Fabric

Hyperledger Fabric was one of the strongest neighboring architectures found.

Its transaction history is immutable and retains both valid and invalid ordered transactions. Endorsement-policy validation occurs within the validation/commit path, and membership infrastructure supports identity and revocation mechanisms. Endorsements and private-data hashes can also serve evidence-like roles.

The comparison nevertheless identified several differences from EASTER.

First, Fabric explicitly maintains a mutable **world state** representing current ledger values. Its immutable transaction history can reconstruct earlier values, but EASTER instead admits immutable State objects and deliberately refuses to designate a canonical current State.

Second, Fabric’s outcome recording is not uniform across all attempted operations in the bounded sense used by the EASTER comparison. Transactions that reach ordering receive valid/invalid outcomes, while proposal or endorsement failures that occur before ordering do not enter the ledger as corresponding durable transaction outcomes.

Third, Fabric’s evidence-like material is generally carried within transaction structures rather than represented as an independently lifecycle-addressable Evidence primitive.

Fabric therefore demonstrates substantial overlap without reproducing the full EASTER boundary.

9.4 Corda

Corda provides a different neighboring architecture.

Its transaction model avoids a single globally broadcast world state and provides strong transaction-local structure, attachments, signatures, and explicit state evolution. On those dimensions, Corda resembles EASTER more closely than Fabric in some decompositions.

The dedicated review nevertheless found its Authority and outcome semantics materially different from EASTER’s.

Transaction signatures establish required participants for particular transactions, while identity revocation operates through a different network/security layer. The reviewed architecture did not demonstrate EASTER’s specific rule that live revocable Authority be validated inside the consequential admission path immediately governing the authoritative write.

Likewise, the investigation did not identify a uniform EASTER-like durable outcome record spanning accepted, rejected, and failed attempts at the same authoritative operation boundary.

Corda therefore overlaps strongly on non-global state and explicit transitions while diverging on the admission boundary EASTER makes central.

9.5 Holochain

Holochain provided useful counterevidence because it makes a substantially different architectural tradeoff.

Its agent-centric design avoids a single canonical global current state. However, the reviewed design explicitly resists externally revocable authorship authority as part of that architecture.

This matters because it suggests that the absence of the EASTER conjunction is not merely terminological. Some neighboring architectures obtain decentralization or non-canonicity by making choices that work against centralized, hard-revocable admission Authority.

EASTER chooses differently. It accepts an authoritative kernel boundary while refusing to let that kernel choose a canonical semantic branch.

The distinction is architectural rather than a claim that one tradeoff is universally preferable.

9.6 Other examined candidates

The closest-prior investigation also considered Ethereum, Certificate Transparency, Apache Kafka, Amazon QLDB, in-toto, Temporal/Cadence, Axon Framework, Git, CRDT-oriented stores, and related systems.

Different subsets of the target properties appear repeatedly.

Ethereum provides durable transaction receipts and immutable history but maintains canonical world state.

Certificate Transparency provides append-only history without defining a canonical trust state, but lacks the examined combination of in-path revocable permission, rejected/failed outcome recording at the same boundary, and separate durable failure diagnostics.

Git and CRDT-oriented systems demonstrate branching or non-canonical continuation particularly clearly but do not combine those properties with the same admission-permission and outcome behavior.

Temporal and related durable-execution systems preserve rich execution and failure histories but treat failure as part of workflow execution semantics rather than EASTER’s separation between accepted work-state, diagnostic failure record, and operation outcome.

in-toto strongly represents signed supporting evidence but is principally a verification/provenance architecture rather than an authoritative transition kernel.

No individual overlap is therefore presented as surprising. The research question concerns their conjunction.

9.7 The surviving behavioral conjunction

Across the closest-prior investigations, four behavioral properties emerged as the narrowest architectural conjunction supported independently by the preserved analyses. They are stated behaviorally here so a prior architecture need not use EASTER’s names or data partitioning to satisfy them:

1. **live revocable permission is validated on the authoritative admission path governing the consequential write;**

2. **attempts that reach that authoritative operation boundary receive a durable outcome distinguishing accepted, rule-rejected, and execution-failed operations, provided the outcome record itself can be persisted;**
3. **execution failure can leave durable diagnostic information without promoting the failed requested work into accepted authoritative work-state; and**
4. **the authoritative substrate deliberately does not designate one admitted work-state as the canonical current semantic state, and permits branching continuation.**

These are behavioral comparison criteria, not requirements that a prior system expose tables or objects named Authority, Receipt, Exception, or State. A differently partitioned architecture that provides equivalent behavior at the same boundary counts as satisfying the corresponding property.

Evidence as a first-class primitive and immutable append-only history remain important parts of EASTER, but the prior-art investigation found closer analogues for those properties individually.

The four-property conjunction is therefore the narrower boundary around the part of the architecture that remained unidentified as a bundle in the systems examined.

The dedicated Fabric, Corda, Holochain, and broader candidate analyses did not identify a system combining all four.

This result supports the following bounded statement:

In the systems and literature examined to date, we did not identify an existing architecture that combines all four behavioral properties at the same consequential continuity boundary.

This is a non-identification claim, not a universal novelty theorem.

9.8 Pre-prior-art provenance of the conjunction

Because a conjunction can be manufactured after examining prior art, we separately tested whether these four properties had been selected retrospectively to occupy an empty region.

The preserved provenance record supports the opposite chronology.

All four properties were present in EASTER artifacts before the dedicated Fabric/Corda/Holochain closest-architecture investigation:

- in-path Authority and revocation behavior existed in the kernel implementation;
- ACCEPTED / REJECTED / FAILED Receipt semantics existed in the schema and implementation;
- durable Exceptions excluded from accepted State were already explicit;
- branching without a kernel-defined canonical current State was already part of the design.

Repository history independently corroborated each of these commitments in implementation or manuscript artifacts predating the closest-prior comparison.

The September 25 manuscript artifact also contained the same coarse architectural commitments before the September 29–30 dedicated prior-art investigation.

This evidence addresses a specific methodological objection:

the four-property conjunction was not constructed after discovering which properties the closest examined architectures lacked.

It does not establish that the conjunction is globally novel.

9.9 Independent decomposition and reconciliation

The two closest-architecture passes used different decompositions and should not be flattened into a single search procedure.

Pax/Muse’s pass used the six-property EASTER combination and pre-identified Fabric as the lead candidate. It identified Fabric as the strongest candidate in that bounded evaluation. Clawde/Sonnet’s later closest-architecture work used an eight-property decomposition and deliberately added Corda and Holochain after Fabric’s canonicity weakness surfaced.

Under an equal-weight reading of Clawde/Sonnet’s properties, Corda compared more closely on some axes, particularly canonicity, evidence separation, and transition explicitness. Pax/Muse’s differently structured pass retained Fabric as the closest candidate.

The subsequent reconciliation found no material contradiction in the underlying primary-source facts. The difference arose from decomposition and weighting.

Rather than hiding that disagreement or converting it into a supposedly objective ranking, the manuscript retains the more defensible result:

Fabric and Corda expose different portions of the EASTER boundary, and neither examined architecture combines the surviving four behavioral properties.

The novelty boundary therefore does not depend on declaring a single architecture the universally “closest” prior system.

9.10 Search stopping and remaining candidate classes

The dedicated closest-architecture search was stopped after the bounded conjunction survived the examined candidate set.

That stopping decision was not treated as proof of exhaustion. A later 4–0 participant vote favored stopping additional dedicated search while preserving the claim as provisional and bounded. The vote records a research-governance decision, not evidence that no counterexample exists.

The preserved rationale was that the search had already crossed several materially different architectural families; differently decomposed passes converged on the same underlying factual gap; and the manuscript claim had been deliberately bounded to the systems and literature examined rather than strengthened into an exhaustive statement.

A particularly relevant untested candidate class remains:

a permissioned distributed architecture combining hard revocable RBAC-like admission with non-canonical or sharded replication.

Such a system could weaken or kill the present non-identification claim if it also provided the bounded outcome recording and durable non-state failure behavior described above at the same boundary.

Pax/Muse’s historical brief also names TUF, OpenLineage, Marquez, DataHub, and Atlas, but its surviving report does not establish whether those candidates were actually examined. They therefore cannot be counted as negative findings from that pass.

These classes should be treated as future prior-art work rather than silently assumed absent.

9.11 Novelty boundary

The related-work investigation supports three different levels of claim, which should not be collapsed.

Supported: individual EASTER mechanisms have substantial prior art.

Supported within the examined set: the four-property behavioral conjunction above was not identified in the systems and literature actually evidenced as examined.

Not established: that no prior system anywhere implements the conjunction, that EASTER is universally novel, that every candidate named in a historical brief was actually screened, or that the conjunction is necessary or optimal.

Accordingly, Draft 2 adopts the bounded formulation:

In the systems and literature examined to date, we did not identify an existing architecture that simultaneously combines live revocable permission validation on the consequential admission path, durable accepted/rejected/failed outcome distinction for attempts reaching that authoritative boundary when persistence succeeds, durable failure diagnostics excluded from accepted work-state, and deliberate refusal by the substrate to designate a canonical current semantic work-state.

This formulation is intentionally falsifiable.

A documented prior architecture satisfying the behavioral conjunction—even with different terminology or internal record types—would narrow or defeat the claim without invalidating the EASTER implementation itself.

That separation between **what the system does** and **what the bounded prior-art investigation has established about its novelty** is essential to the evidentiary boundary of this paper.

10 Limitations and Threats to Validity

The results in this paper are bounded by the size and selection of the architectural corpus, uneven archival completeness, the observational nature of several findings, the absence of formal minimality proofs, and the scope of the prior-art investigation.

These limitations constrain the claims that can be made from the present work.

10.1 Small and non-random architectural corpus

The comparative study examined six mature agent architectures:

- Hermes Agent;
- OpenClaw;
- LangGraph;
- Anthropic Claude Agent SDK;
- OpenAI Agents SDK; and
- Google Antigravity.

These systems do not constitute a statistical sample of agent architectures.

They were selected within a research program already concerned with continuity, representation, memory, and agent-system behavior. The corpus is therefore biased toward comparatively mature systems likely to contain rich continuity mechanisms.

The observed EASTER and DRC results should not be generalized statistically to agent systems as a population.

The absence of a demonstrated EASTER GAP in four of the six systems likewise should not be interpreted as an estimate of the prevalence of EASTER-like properties in agent architectures generally.

10.2 Uneven observability

The inspected systems differed substantially in how much of their architecture was available for review.

Open-source systems permitted source-oriented inspection. Managed systems exposed a narrower boundary consisting primarily of public documentation, interfaces, and behavior visible to the research program.

A GAP therefore means that a property was not demonstrated within the inspected boundary. It does not establish that no corresponding internal mechanism exists.

Conversely, COVERED means that sufficient evidence was identified within the review boundary to support the particular disposition. It does not establish implementation-wide correctness, security, or completeness.

Comparisons across systems must therefore account for observability asymmetry.

10.3 Uneven archival completeness

The publication archive is incomplete at the primitive level for portions of the six-system review.

Some systems retain detailed primitive sweeps and whole-system composition findings. Others retain only aggregate results. Exact source commits, versions, individual DRC receipts, or primitive matrices are missing for portions of the corpus.

The paper does not reconstruct those missing classifications from aggregate counts or present-day recollection.

This preserves provenance fidelity but limits reproducibility.

The comparative table should consequently be understood as a report of the strongest preserved review state currently available, not as a normalized benchmark dataset with equal evidentiary depth for every system.

10.4 Review evolution and retrospective rationale

The reverse-review method evolved during the research program.

Not every methodological decision was preregistered or frozen before the corresponding review. In particular, the rationale for stopping the initial six-system corpus was reconstructed after the stopping point.

Draft 2 therefore distinguishes procedures preserved as actually performed from explanations developed retrospectively.

Retrospective explanation may help interpret the research process, but it cannot acquire the evidentiary status of a preregistered decision merely because it is plausible.

10.5 Human–AI research dependence

The work was produced through substantial collaboration among a human researcher and multiple AI systems.

AI collaborators participated in architectural analysis, implementation, adversarial review, literature investigation, experimental design, provenance reconstruction, and manuscript development.

Some review rounds intentionally isolated AI responses to reduce direct reviewer-to-reviewer anchoring. Such isolation does not create independent scientific replication.

The collaborators shared a research program, overlapping vocabulary, common artifacts, and human-mediated context. Their errors may therefore be correlated even when individual responses were produced separately.

Agreement among the collaborators is treated as evidence of response convergence, not independent validation.

Where disagreement mattered, the intended resolution mechanism was inspection of primary evidence rather than majority vote.

10.6 No proof of EASTER minimality

No preserved review artifact records a CHALLENGED disposition against an EASTER primitive in the recovered corpus.

That observation does not establish that the primitive set is minimal.

A seventh candidate, Invariant, was explicitly considered during the LangGraph analysis and rejected as unnecessary for representing the execution that actually occurred. This is evidence that

at least one candidate extension was tested.

It is not a proof that no smaller basis or alternative decomposition can represent the same continuity properties.

The present paper therefore proposes EASTER as a compact six-primitive model supported by the examined evidence, not as a mathematically minimal basis.

10.7 No proof of universal sufficiency

No reviewed system demonstrated a need for an additional EASTER primitive under the applied lens.

This does not establish universal sufficiency.

The six-system corpus may simply fail to contain architectures or consequential workflows that expose a missing category.

Future systems may reveal continuity properties that cannot be adequately represented through Evidence, Authority, State, Transition, Exception, and Receipt.

A demonstrated case of that kind should be treated as evidence against the present primitive set rather than forced into an existing category.

10.8 DRC remains a candidate vocabulary

The DRC overlay classified Distinction, Relation, and Constraint as COVERED for all six architectures.

No DRC-negative or borderline specimen was observed in the selected corpus. The recovered DRC classifications are also incomplete at the primitive-receipt level for portions of the corpus, so the historical classifications cannot all be independently reconstructed from the surviving package.

The six systems were selected as mature, representation-rich agent architectures for the EASTER investigation, not as specimens designed to probe DRC’s discriminative lower bound. The uniform COVERED result therefore does not establish whether DRC would discriminate among less mature or representationally weaker systems.

No ablation experiment demonstrated that removing one of the three categories necessarily destroys functional interpretability.

No formal proof establishes that DRC is minimal, universally necessary, or sufficient for meaning.

The observed 3/3 coverage across six mature systems is therefore compatible with several interpretations: DRC may identify highly general representational structure, mature agent architectures may have converged on structures satisfying DRC, the categories may be too broad to discriminate strongly, the corpus may be biased toward systems rich enough to satisfy them, or some combination of these may be true.

The present paper cannot distinguish among those possibilities and therefore does not establish DRC’s discriminative lower bound.

This limitation does not alter the narrower observation that, under the classifications applied in this corpus, DRC coverage did not entail zero EASTER GAPS. That statement reports the

relationship among the preserved classifications; it is not a claim that DRC’s lower bound has been independently established.

DRC should consequently be treated as a candidate analytical vocabulary supporting the userland/continuity boundary, not as an established universal ontology.

10.9 Projection fidelity

EASTER preserves the consequential representation admitted to the kernel.

It cannot recover information that userland failed to project.

A system may therefore satisfy EASTER’s continuity requirements while preserving an incomplete, distorted, or semantically inadequate representation.

This limits what continuity alone can guarantee.

EASTER does not establish semantic truth, completeness of userland representation, correctness of a model’s reasoning, or adequacy of the application’s interpretation.

Projection fidelity remains an external responsibility.

10.10 Operational provenance and retrieval limitation

The contribution-provenance workflow described in Section 8 occurred during real preparation of this manuscript and was not designed prospectively as a controlled experiment.

Two later retrievals tested a narrower question raised by the independent cold review: whether consequential publication history preserved in EASTER could be recovered independently from the stored records. Pax/Muse and Hermes froze separate reports before seeing one another’s result and used different retrieval capabilities. Both recovered materially coherent publication-provenance histories, but neither result establishes a complete or mechanically traversable end-to-end graph.

The limitations are structural to the evidence package rather than contradictions in the recovered histories. Some relationships are explicit stable-ID links; others are embedded in payload fields or inferred from shared project, pull-request, commit, subject, and chronology information. Hermes could enumerate the authenticated API and therefore recovered substantially more history than Pax/Muse, whose **agent** role was limited to traversal from known identifiers. Later publication work is also represented predominantly through Evidence records rather than one continued State/Transition chain.

The retrieval result therefore supports historical preservation and bounded recoverability. It does not establish self-indexing graph semantics, canonical history selection, complete autonomous reconstruction, or independent verification of external GitHub assertions preserved inside EASTER payloads.

The workflow and retrievals were performed by systems involved in the EASTER research program. Independent users applying EASTER to consequential workflows they did not design would provide stronger evidence.

10.11 Reference-implementation dependence

Several demonstrated properties depend on the current Python/SQLite reference implementation.

SQLite transactions, schema constraints, triggers, explicit Authority ledgers, and particular kernel operations are implementation mechanisms used to realize EASTER’s claimed invariants.

A different implementation could claim compatibility while implementing those mechanisms differently.

The present study does not provide a formal conformance suite establishing exactly which observable behaviors another implementation must satisfy to qualify as EASTER-compatible.

Without such a suite, some boundary remains between the conceptual model and implementation-specific behavior.

Developing implementation-independent conformance tests is therefore future work.

10.12 Prior-art search is bounded

The related-work investigation crossed multiple architectural families and included dedicated analysis of several strong neighboring systems.

It was not exhaustive.

The paper’s novelty language is therefore intentionally limited to systems and literature examined to date.

A prior architecture satisfying the four-property conjunction identified in Section 9 would narrow or defeat the present non-identification claim.

In particular, the research has not exhausted the candidate class of permissioned distributed architectures combining hard revocable admission control with non-canonical or sharded replication.

The novelty claim should therefore remain falsifiable and updateable as additional prior art is found.

10.13 Conjunction provenance does not prove novelty

The preserved pre-prior-art record provides evidence that EASTER’s four-property conjunction existed before the dedicated closest-prior investigation.

This addresses retrospective gerrymandering of the conjunction.

It does not prove that the conjunction had never previously appeared elsewhere.

Chronology establishes that the researchers did not construct the conjunction after discovering the examined prior-art gap; it does not establish historical priority over unknown work.

These are separate claims and should remain separate.

10.14 Lack of independent replication

The present findings have not been independently replicated by researchers outside the development and review process.

This limitation applies both to the six-system architectural analysis and to the reference implementation.

Independent replication could challenge primitive classifications, identify overlooked prior art, expose missing continuity categories, or demonstrate that some current OPEN findings should be resolved differently.

Such disagreement would be informative rather than anomalous.

The review framework is intended to preserve enough specificity that competing classifications can be compared against underlying evidence.

10.15 Threat model boundary

EASTER is not presented as a complete security architecture.

The model addresses continuity of consequential work across computational change. Authentication mechanisms, credential issuance, transport security, secret management, network security, identity proofing, application policy, and payload semantics remain outside the six primitives except insofar as their consequential outcomes cross the kernel boundary.

Authority records can preserve who was permitted to perform an operation under the implemented rules.

They do not prove that the external identity mechanism was uncompromised or that the policy granting that Authority was normatively correct.

Similarly, immutable provenance can preserve a malicious or mistaken event faithfully.

Continuity should therefore not be confused with trustworthiness, safety, or correctness.

10.16 Scope of the present evidence

Taken together, the present evidence supports a bounded conclusion.

It shows that:

- a six-primitive continuity model can be instantiated as a working reference kernel;
- the model can be applied as an architectural review lens;
- the examined corpus contains both substantial overlap with EASTER and bounded demonstrated gaps;
- DRC coverage did not entail zero EASTER Gaps under the applied classifications in that corpus;
- the reference implementation supported one real multi-participant provenance workflow;
- consequential portions of that publication history were independently recoverable from EASTER records by two retrievers using different access and traversal methods; and
- the closest-prior investigation did not identify the surviving four-property conjunction in the systems and literature examined.

The evidence does not establish universal sufficiency, minimality, semantic correctness, statistical prevalence, exhaustive novelty, complete self-indexing provenance reconstruction, or independent replication by researchers outside the development process.

Those boundaries are part of the result rather than qualifications to be removed from it.

11 Discussion and Research Implications

The preceding results support EASTER as a working continuity architecture and analytical model under the boundaries stated in Section 10.

They also suggest broader research questions.

Those questions should not be confused with demonstrated results. In particular, this study does not establish universal sufficiency of the six primitives, semantic portability across arbitrary runtimes, or a general theory of intelligent systems.

What it provides is a concrete architecture from which those questions can be tested.

11.1 Continuity as a mechanically inspectable record

EASTER began as a continuity problem: how can consequential intelligent work survive changes in models, agents, processes, sessions, machines, and runtimes without depending on the private state of the system that originally performed the work?

The resulting architecture suggests a useful reframing.

Continuity is not merely the persistence of information.

For consequential work, later participants may need to determine:

- what evidence existed;
- who possessed authority;
- what state was admitted;
- what transition occurred;
- what failed or was refused; and
- what durable outcome was recorded.

These questions correspond directly to EASTER's six primitives.

In a deliberately mechanical sense, EASTER can therefore be described as an **accountability machine**: a substrate designed to preserve inspectable attribution and outcome structure without requiring access to the originating runtime's private computational state.

The phrase does **not** mean moral, legal, institutional, or democratic accountability. EASTER does not establish identity assurance, legitimacy of the policy that granted Authority, fairness, due process, enforcement, remedy, or whether an action was morally, legally, or semantically correct. Those require mechanisms and judgments outside the kernel.

The kernel provides records that an external accountability process may inspect; it does not itself constitute that process.

11.2 Observation is not accountability

The distinction between persistence and accountability is important.

A system may record extensive logs, traces, messages, prompts, model outputs, and telemetry while still failing to preserve the structure necessary to determine which records were consequential, authorized, accepted, rejected, or failed.

Observation provides information.

Mechanical accountability requires that consequential records can be attributed and inspected in a bounded way. Broader accountability additionally requires external governance capable of contesting records and making that contest consequential.

The present architecture addresses only the mechanical layer.

Evidence may exist without becoming State.

An attempted action may generate a Receipt without being authorized.

An Exception may remain durable without becoming accepted State.

A branch may remain historically real without becoming the kernel's chosen current truth.

These distinctions are what allow EASTER to preserve consequential history without requiring the substrate to decide semantic truth.

11.3 Authority without semantic sovereignty

One of the more unusual combinations in EASTER is the coexistence of strong admission Authority with refusal to designate a canonical semantic branch.

The kernel is authoritative about whether an operation may cross its boundary.

It is intentionally non-authoritative about which valid branch represents the one correct interpretation of the world.

This separates two forms of power that are often conflated:

admission authority — whether a consequential operation is permitted to occur;

and

semantic sovereignty — which interpretation, branch, or state should be treated as the authoritative meaning of the work.

EASTER implements the first while declining the second.

The prior-art investigation suggests that this combination contributes materially to EASTER's architectural distinctiveness, although the bounded search does not establish its global novelty.

The distinction may also be useful beyond EASTER. Systems that require strong governance need not necessarily require a substrate that chooses semantic truth.

11.4 Refusal and failure as first-class history

The reference implementation treats ACCEPTED, REJECTED, and FAILED outcomes as historically meaningful for operations that reach the receipt-capable kernel path and whose outcome records can be persisted.

This has a consequence for continuity.

A rejected operation is not simply absence.

A failed operation is not equivalent to an operation that never occurred.

Both may influence later reasoning about what was attempted, what Authority existed, what evidence was available, or why subsequent work took a different path.

Preserving these outcomes therefore increases the reconstructability of consequential work without requiring unsuccessful operations to mutate accepted State.

This claim is bounded to operations that reach the receipt-capable kernel path. Authentication or other deployment-layer events that terminate before that boundary are outside the kernel's Receipt semantics.

Future work should test whether bounded outcome preservation materially improves recovery after more severe runtime changes.

11.5 Branching as preserved disagreement

EASTER's refusal to designate a canonical current State also changes the treatment of disagreement.

Branching permits multiple accepted successor States to descend from a common predecessor without requiring the kernel to decide which branch is semantically correct.

This can represent alternative hypotheses, competing interpretations, independent participant accounts, or divergent plans while preserving their shared ancestry.

The kernel can therefore preserve disagreement structurally.

It does not resolve disagreement procedurally.

Contestability, adjudication, reconciliation, and consequences remain userland responsibilities.

This distinction matters for systems involving multiple intelligent participants. Preserving a counter-record is not equivalent to providing due process, and inspectability alone does not ensure that a challenge can affect an outcome.

11.6 Semantic work-state portability and its oracle

The architecture motivates a stronger experimental hypothesis:

if the consequential state of intelligent work is faithfully projected through EASTER, a different EASTER-capable runtime may be able to continue that work without access to the originating runtime's private state.

This is distinct from checkpoint portability.

Checkpoint portability attempts to reproduce enough runtime-specific state to resume substan-

tially the same computational process.

It is also distinct from context portability, in which messages, prompts, summaries, or memory are transferred between systems.

The proposed target is **semantic work-state portability**.

The receiving system need not recreate the original runtime. It must produce an acceptable next consequential action from the preserved work-state under the same externally specified task constraints.

This hypothesis has not been demonstrated by the present six-system review or the attribution case study. A future test therefore needs an oracle defined **before** migration rather than judging continuation after seeing Runtime B’s answer.

For a portability trial, the experiment should specify prospectively:

1. **task contract** — the goal, admissible actions, invariants, and stopping conditions that define the work independently of either runtime;
2. **migration cut** — the exact point at which Runtime A loses authority to continue and the EASTER projection is frozen for transfer;
3. **information boundary** — Runtime B receives the designated EASTER records and declared public task inputs, but not Runtime A’s private conversation, hidden memory, caches, scratch state, or implementation-specific checkpoint;
4. **acceptable-successor set** — where multiple next actions are valid, the oracle defines a set or predicate of acceptable consequential successors rather than requiring byte-for-byte reproduction of Runtime A’s hypothetical next output;
5. **invalid-successor conditions** — violations of task invariants, Authority, preserved evidence constraints, or required ancestry count as failures even if the resulting output appears useful;
6. **baselines** — compare EASTER transfer against at least a no-transfer condition and a context-transfer baseline such as a conventional summary or transcript package; and
7. **measures** — record continuation validity, consequential errors, information transferred, recovery time or work required, and any human intervention needed before the next accepted transition.

The task contract, acceptable-successor predicate, invalid-successor conditions, and adjudication procedure should be frozen before the migration trial. Where practicable, the person or system defining those criteria and the evaluator judging trial success should be independent of the EASTER development team or blinded to treatment condition. If that independence is unavailable, the resulting limitation should be reported explicitly rather than treating contributor agreement as independent validation.

Under this oracle, **correct continuation** means that Runtime B can produce a successor satisfying the predeclared task contract and continuity constraints from the permitted transfer package. It does not mean that Runtime B must make the same stylistic or internal reasoning choices Runtime A would have made.

A stronger multi-hop experiment would repeat the same rule across:

Runtime A $\rightarrow$ EASTER $\rightarrow$ Runtime B $\rightarrow$ EASTER $\rightarrow$ Runtime C

Success across heterogeneous runtimes would provide evidence that EASTER captures portable work structure rather than merely recording one implementation. Failure would be equally informative because it could expose missing primitives, inadequate projection fidelity, an underspecified task oracle, or necessary userland structure that EASTER does not preserve.

11.7 The macroscopic-variable hypothesis

The portability question motivates a broader research hypothesis.

Modern intelligent systems contain enormous quantities of microscopic computational state: activations, token histories, caches, hidden representations, scheduler state, process memory, implementation details, and runtime-specific artifacts.

Much of that state may be unnecessary for preserving the continuity of consequential work.

This suggests the hypothesis:

Evidence, Authority, State, Transition, Exception, and Receipt may be sufficient macroscopic variables for preserving consequential intelligent work across microscopic computational change.

The analogy is intentionally structural rather than physical.

A macroscopic description is useful when it preserves the variables needed to reason about system behavior without reproducing every microscopic degree of freedom.

EASTER may play such a role for consequential intelligent work.

The present paper does not prove this hypothesis.

The six-system reviews establish that the primitives form a useful architectural lens across heterogeneous systems. The reference implementation establishes that they can be instantiated. The provenance case establishes one operational use.

None establishes that the six variables are sufficient for arbitrary cross-runtime continuation.

That question requires direct portability experiments using a prospectively defined oracle such as Section 11.6.

11.8 Compounding capability without centralizing runtime state

If semantic work-state portability proves feasible, it could support a different architecture for multi-system intelligence.

Instead of requiring one model, agent framework, memory service, or orchestration runtime to own the complete history of a project, heterogeneous systems could operate against a shared continuity substrate.

Individual runtimes could remain replaceable.

Consequential work could outlive particular models.

Specialized systems could contribute without becoming the permanent owner of project memory.

Failures or migrations would not necessarily require reconstructing the exact originating runtime.

This possibility aligns with a broader design principle:

intelligence should compound capability, not centralize power.

EASTER alone does not guarantee that outcome.

A shared continuity substrate can itself become a point of control. Authority policy, deployment topology, access mechanisms, governance, and userland design determine how power is actually distributed.

The architecture merely makes it possible to separate continuity from ownership by a particular intelligence runtime.

11.9 EASTER as a slow layer

The implementation also suggests a useful systems distinction between fast-changing intelligence infrastructure and slower continuity semantics.

Models, harnesses, tools, agent frameworks, memory systems, orchestration strategies, and user interfaces are likely to change rapidly.

A continuity layer benefits from changing more slowly.

Under this view:

the harness is userland; EASTER is the slow layer.

The phrase describes an architectural aspiration rather than an invariant demonstrated by this study.

The kernel should contain only semantics that must remain stable across changing intelligence runtimes. Application meaning, model behavior, authentication mechanisms, orchestration, and policy should remain outside unless evidence demonstrates that they are necessary continuity semantics.

This creates a conservative evolution rule for the primitive set:

do not add a primitive because a feature is useful; add one only when consequential continuity cannot be represented without it.

The absence of a demonstrated CHALLENGED disposition in the initial corpus provides some support for restraint, but not proof that the six-primitive boundary is final.

11.10 Falsification program for DRC

The present corpus cannot distinguish a genuinely general representational vocabulary from categories broad enough to classify almost any mature system. Future DRC work should therefore include specimens selected to produce negative and borderline cases, not only additional mature architectures expected to satisfy all three categories.

A useful test matrix would include deliberately reduced representations:

- **Distinction-negative:** records whose purported entities cannot be stably distinguished from one another or from their attributes;

- **Relation-negative:** distinguishable records presented without recoverable structural relationships among them;
- **Constraint-negative:** distinguishable, related records for which no rule, admissibility condition, invariant, or other limiting structure governs the represented relationships; and
- **borderline specimens:** structures in which a category is only weakly or implicitly recoverable, forcing reviewers to state what evidence is sufficient for COVERED versus OPEN.

The test should freeze category definitions and disposition rules before classification. Independent reviewers should classify the same specimens without seeing one another's results. If deliberately negative specimens still receive 3/3 coverage because reviewers can always reinterpret some feature as Distinction, Relation, or Constraint, that would be evidence that DRC is too permissive to function as a discriminating analytical vocabulary.

Conversely, stable negative classifications and informative borderline disagreements would strengthen the claim that the categories have falsifiable content.

This program tests discriminability; it would still not establish that DRC is a universal or minimal theory of meaning.

11.11 Toward an implementation-independent EASTER contract

The Python/SQLite kernel is one realization of EASTER, not the definition of every possible conforming implementation. A compact behavioral contract can nevertheless make the present architecture more machine-checkable without requiring another implementation to copy its schema.

At minimum, a candidate EASTER-compatible kernel should expose observable behavior satisfying the following conditions:

Admission preconditions

- an ordinary consequential transition references an existing admitted predecessor State;
- the acting identity has a live, non-revoked Authority grant when the operation requires Authority;
- the proposed successor State and Transition satisfy structural validity before admission; and
- Evidence, when referenced, is independently addressable rather than becoming State merely by being supplied.

Accepted-operation postconditions

- the new State and its Transition are durably admitted atomically;
- previously admitted State is not mutated in place;
- branching from an admitted predecessor is permitted without the kernel designating a canonical semantic successor; and

- the new State, its Transition, and the ACCEPTED Receipt identifying the admitted operation commit atomically as one accepted authoritative outcome.

Rule-rejection postconditions

- a rule-rejected operation does not mutate accepted State;
- where the request has reached the receipt-capable kernel path and persistence succeeds, a REJECTED Receipt durably records the outcome; and
- rejection does not require fabrication of an Exception representing an execution failure.

Execution-failure postconditions

- partial authoritative mutation from the attempted operation is unwound;
- after the attempted authoritative transaction has unwound, a separate diagnostic-recording transaction durably persists a FAILED Receipt and linked Exception describing the failed attempt when that diagnostic transaction succeeds; and
- if that diagnostic transaction itself cannot persist, the implementation must not expose phantom durable failure records.

Authority postconditions

- grant, revoke, and revoke-all operations preserve an inspectable Authority history rather than silently rewriting prior grants; and
- a revoked or expired grant cannot authorize a later protected transition under the same grant.

These statements are a manuscript-level behavioral contract, not yet a complete formal specification or conformance suite. They intentionally describe observable semantics rather than SQLite tables, triggers, or Python call structure.

Future conformance work should encode them as executable tests against at least one independently implemented kernel. A second implementation that passes the behavioral suite while using different storage and interface mechanisms would provide stronger evidence that EASTER is an architecture rather than merely the current codebase.

11.12 Future experimental program

The next research phase should prioritize falsification over additional descriptive confirmation.

Several experiments follow directly from the current limitations.

Cross-runtime continuation. Test semantic work-state portability between heterogeneous agent runtimes using the prospectively frozen oracle, information boundary, baselines, and failure metrics in Section 11.6.

Primitive ablation. Remove or collapse individual EASTER primitives and measure which continuity properties become unrecoverable.

DRC-negative specimens. Execute the negative/borderline program in Section 11.10 and test whether the vocabulary remains discriminative under frozen classification rules.

Projection-fidelity failures. Supply incomplete or distorted userland projections and determine which failures EASTER can detect and which it faithfully preserves.

Conformance testing. Turn the behavioral contract in Section 11.11 into executable implementation-independent tests and run them against a second kernel implementation.

Independent reverse review. Have researchers outside the development process reproduce or challenge the six-system classifications from primary sources.

Prior-art falsification. Search specifically for architectures in the remaining high-risk candidate classes identified in Section 9.

Adversarial Authority testing. Evaluate revocation races, replay, stale credentials, concurrent transitions, and external-effect boundaries under deliberately hostile conditions.

The goal of these experiments should not be to protect the current architecture.

It should be to discover where it breaks.

11.13 Research implication

The strongest implication of the present work is therefore not that six primitives solve continuity universally.

It is that consequential continuity can be treated as an architectural object in its own right.

The originating intelligence need not necessarily remain present.

The originating runtime need not necessarily remain alive.

The complete microscopic computational history need not necessarily remain reproducible.

What may need to survive is the consequential structure of the work.

EASTER provides one concrete proposal for what that structure consists of.

Whether that proposal is sufficient across genuinely heterogeneous intelligent systems is now an experimentally testable question.

12 Conclusion

Intelligent work increasingly crosses boundaries between models, agents, processes, sessions, machines, and runtimes. Preserving that work requires more than retaining messages or reconstructing the private computational state of the system that originally performed it.

This paper proposed **EASTER** — **E**vidence, **A**uthority, **S**tate, **T**ransition, **E**xception, **a**nd **R**eceipt — as a six-primitive model for preserving the continuity of consequential intelligent work across computational change.

The model separates six questions:

- **Evidence:** what supports a consequential claim or action?
- **Authority:** who or what was permitted to act?
- **State:** what immutable consequential representation was admitted?
- **Transition:** what explicit change connected admitted States?
- **Exception:** what consequential failure or diagnostic condition occurred without becoming accepted State?
- **Receipt:** what durable kernel outcome records an attempted operation that reached the receipt-capable path and whose outcome record could be persisted?

A Python/SQLite reference kernel demonstrates that these distinctions can be implemented while keeping semantic interpretation, authentication, application policy, orchestration, and model behavior outside the continuity substrate.

For operations that reach its receipt-capable path, the kernel admits immutable State, permits branching, validates revocable Authority in the consequential admission path, preserves Exceptions outside accepted State, and distinguishes ACCEPTED, REJECTED, and FAILED outcomes without designating a canonical semantic branch. Requests stopped before the kernel boundary and failures that prevent outcome-record persistence lie outside that Receipt guarantee.

We also preserve a historical reverse-review of six mature agent architectures as a recovered methodological record. Its surviving archive retains a frozen system-level aggregate and substantial primitive-level material, but it does not preserve the complete original classification package, exact source pins for every target, or enough detail to reproduce every historical disposition. The numeric aggregate is therefore retained in Section 6 and Appendix A as part of the research record rather than presented here as a principal reproducible empirical result.

No preserved review artifact records a CHALLENGED disposition against an EASTER primitive in the recovered corpus. That archival observation supports continued investigation of the six-primitive model but is not evidence that every primitive was affirmatively stress-tested in every system, and it does not establish minimality or universal sufficiency.

A complementary DRC overlay — Distinction, Relation, and Constraint — records all three DRC categories as COVERED in all six systems while some systems retain EASTER GAPS. Because those classifications inherit the same archival limitations, this is best treated as an observed asymmetry in the recovered coding record and a motivation for prospective testing, not as an established architectural non-entailment result.

The reference implementation was also used during preparation of this manuscript to preserve a real multi-participant contribution-provenance workflow. That case demonstrated operational preservation of independent claims, corrections, repository integration, and attributable kernel-admitted outcomes while leaving semantic truth and publication judgment outside the kernel.

A dedicated closest-prior investigation found substantial overlap with existing architectural traditions. Individual EASTER mechanisms are not claimed as novel.

Within the systems and literature examined to date, however, we did not identify an existing architecture that simultaneously combines the following behavioral properties:

1. revocable admission permission validated as part of the consequential state-changing operation rather than only as advisory or retrospective metadata;
2. durable ACCEPTED, REJECTED, and FAILED outcome records for operations that reach the authoritative receipt-capable path, when the corresponding outcome record can itself be persisted;
3. durable diagnostic failure records kept distinct from accepted consequential State; and
4. deliberate support for multiple valid State successors without requiring the substrate to designate one canonical current semantic State.

The provenance record independently establishes that this conjunction existed in EASTER before the dedicated closest-prior investigation. This supports the chronology of the architectural claim but does not establish exhaustive historical novelty.

The broader hypothesis remains unproven:

Evidence, Authority, State, Transition, Exception, and Receipt may be sufficient macroscopic variables for preserving consequential intelligent work across microscopic computational change.

That hypothesis now admits a direct experimental test with a prospectively defined continuation oracle.

If consequential work is faithfully projected through EASTER, can a heterogeneous runtime produce an acceptable next consequential successor under predeclared task and continuity constraints after access to the originating runtime’s private state, conversation, memory, and implementation machinery has been removed?

Success would provide evidence for **semantic work-state portability**.

Failure could expose missing primitives, inadequate projection fidelity, an underspecified continuation oracle, or continuity structure that the current model does not capture.

Either result would advance the question.

EASTER therefore should not be treated as a finished theory of intelligent continuity.

It is a concrete architecture, a working implementation, a bounded body of evidence, and a falsifiable proposal about what consequential work may need in order to survive the intelligence that created it.

References

- [1] E. Androulaki, A. Barger, V. Bortnikov, C. Cachin, K. Christidis, A. De Caro, D. Enyeart, C. Ferris, G. Laventman, Y. Manevich, S. Muralidharan, C. Murthy, B. Nguyen, M. Sethi, G. Singh, K. Smith, A. Sorniotti, C. Stathakopoulou, M. Vukolić, S. W. Cocco, and J. Yellick, “Hyperledger Fabric: A Distributed Operating System for Permissioned Blockchains,” *Proceedings of the Thirteenth EuroSys Conference (EuroSys ’18)*, 2018. DOI: 10.1145/3190508.3190538. Publication record: <https://research.ibm.com/publications/hyperledger-fabric-a-distributed-operating-system-for-permissioned-blockchains>

- [2] Hyperledger Fabric contributors, *Hyperledger Fabric Documentation*. Linux Foundation / Hyperledger. <https://hyperledger-fabric.readthedocs.io/> (accessed 2026-10-01).
- [3] M. Hearn and R. G. Brown, “Corda: A distributed ledger,” technical whitepaper, version 1.0, 20 Aug. 2019. Authoritative R3-hosted copy: <https://docs.r3.com/en/pdf/corda-technical-whitepaper.pdf>
- [4] Holochain, *Developer Documentation: Working with Data, the DHT, and Validation*. <https://developer.holochain.org/> (accessed 2026-10-01).
- [5] P. Groth and L. Moreau, eds., “PROV-Overview: An Overview of the PROV Family of Documents,” W3C Working Group Note, 30 Apr. 2013. <https://www.w3.org/TR/prov-overview/>
- [6] L. Moreau and P. Missier, eds., “PROV-DM: The PROV Data Model,” W3C Recommendation, 30 Apr. 2013. <https://www.w3.org/TR/prov-dm/>
- [7] D. Hardt, ed., “The OAuth 2.0 Authorization Framework,” RFC 6749, Internet Engineering Task Force, Oct. 2012. <https://www.rfc-editor.org/rfc/rfc6749.html>
- [8] J. B. Dennis and E. C. Van Horn, “Programming Semantics for Multiprogrammed Computations,” *Communications of the ACM*, vol. 9, no. 3, pp. 143–155, Mar. 1966. DOI: 10.1145/365230.365252.
- [9] H. M. Levy, *Capability-Based Computer Systems*. Digital Press, 1984. ISBN 0-932376-22-3. Author-hosted edition: <https://homes.cs.washington.edu/~levy/capabook/>
- [10] Temporal Technologies, *Temporal Platform Documentation: Durable Execution*. <https://docs.temporal.io/temporal> (accessed 2026-10-01).
- [11] Apache Software Foundation, *Apache Kafka Documentation: Design*. <https://kafka.apache.org/43/design/> (accessed 2026-10-01).
- [12] AxoniQ, *Axon Framework Reference: Events and Event Store Infrastructure*. <https://docs.axoniq.io/axon-framework-reference/latest/events/infrastructure/> (accessed 2026-10-01).
- [13] Event Store, *EventStoreDB Documentation*. <https://developers.eventstore.com/> (accessed 2026-10-01).
- [14] Git project, *Git Documentation*. <https://git-scm.com/docs> (accessed 2026-10-01).
- [15] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, “Conflict-Free Replicated Data Types,” *13th International Symposium on Stabilization, Safety, and Security of Distributed Systems (SSS 2011)*, LNCS 6976, pp. 386–400, 2011. DOI: 10.1007/978-3-642-24550-3_29.
- [16] B. Laurie, A. Langley, and E. Kasper, “Certificate Transparency,” RFC 6962, Internet Engineering Task Force, June 2013. <https://www.rfc-editor.org/rfc/rfc6962.html>. (RFC 6962 is obsolete; see RFC 9162 for the current specification. It is retained here because the historical related-work investigation referenced the earlier Certificate Transparency specification family.)
- [17] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappel, “in-toto: Providing farm-to-table guarantees for bits and bytes,” *28th USENIX Security Symposium (USENIX Security ’19)*, pp. 1393–1410, 2019. <https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias>

- [18] G. Wood, “Ethereum: A Secure Decentralised Generalised Transaction Ledger,” Ethereum Yellow Paper. Project repository: <https://github.com/ethereum/yellowpaper> (historical architectural reference; the repository notes that the Yellow Paper is not current beyond the Shanghai-era specification).
- [19] Amazon Web Services, *Amazon Quantum Ledger Database (QLDB) Developer Guide*. Historical documentation URL: <https://docs.aws.amazon.com/qldb/latest/developerguide/> (AWS ended QLDB support on 2025-07-31; this documentation URL was no longer served when rechecked on 2026-10-01. The entry is retained as a historical architectural reference rather than replaced with an unrelated current source.)
- [20] Nous Research, *Hermes Agent Documentation*. <https://hermes-agent.nousresearch.com/docs/> (publication-navigation reference; accessed 2026-10-01).
- [21] OpenClaw contributors, *OpenClaw Documentation: Sessions and Memory*. <https://docs.openclaw.ai/session> (publication-navigation reference; accessed 2026-10-01).
- [22] LangChain, *LangGraph Documentation*. <https://docs.langchain.com/oss/python/langgraph/> (publication-navigation reference; accessed 2026-10-01).
- [23] Anthropic, *Claude Agent SDK / Agent Runtime Documentation*. <https://platform.claude.com/docs/> (publication-navigation reference; accessed 2026-10-01).
- [24] OpenAI, *OpenAI Agents SDK Documentation*. <https://developers.openai.com/api/docs/guides/agents/sdk> (publication-navigation reference; accessed 2026-10-01).
- [25] Google, *Google Antigravity Documentation*. <https://antigravity.google/docs/> (publication-navigation reference; accessed 2026-10-01).
- [26] N. Woolen, *EASTER reference implementation*, Git commit 8bf422747836c96233f8dc11d4b11d1a61c832c1, 29 Sep. 2026. <https://github.com/witsbi/easter/tree/8bf422747836c96233f8dc11d4b11d1a61c832c1>
- [27] N. Woolen, *EASTER Draft 2 accepted post-remediation baseline and research archive*, Git commit 18ffc91e6550a6df7b41f335dd1879ec0002d111, 1 Oct. 2026. <https://github.com/witsbi/easter-paper/tree/18ffc91e6550a6df7b41f335dd1879ec0002d111>
- [28] Ori, “Frozen Review Recovery,” archived EASTER research artifact, 29 Sep. 2026; archived by Pax/Muse without reclassification, pinned in [27] at `paper/archive/frozen-review-recovery-2026-09-29.md`.
- [29] Pax/Muse, “Prior-Art First-Party Account,” archived EASTER research artifact, 1 Oct. 2026, pinned in [27] at `paper/archive/pax-prior-art-first-party-account-2026-10-01.md`.
- [30] Clawde/Sonnet, “Prior-Art Methodology Account,” archived EASTER research artifact, 1 Oct. 2026, pinned in [27] at `paper/archive/clawde-prior-art-methodology-account-2026-10-01.md`.
- [31] Pax/Muse and Clawde/Sonnet, “Closest-Architecture Reconciliation,” archived EASTER research artifact, 30 Sep. 2026, pinned in [27] at `paper/archive/clawde-pax-closest-architecture-reconciliation-2026-09-30.md`.

- [32] Pax/Muse, “Pre-Prior-Art Conjunction Provenance,” archived EASTER research artifact, 29 Sep. 2026; compiled at Ori’s direction via Mercury relay, pinned in [27] at `paper/archive/pre-prior-art-conjunction-provenance-2026-09-29.md`.
- [33] Clawde/Sonnet, “Independent Adversarial Review of Draft 2,” frozen EASTER research artifact, 1 Oct. 2026; reviewed subject commit `6386f9927c6111205ca0b188c10c0c32353f5b50`, pinned at artifact commit `4cb991f87f11a7efe8de81ad35aaf20a6901bae9` (merge of PR #14) at `paper/reviews/clawde-draft2-adversarial-review-2026-10-01.md`.
- [34] Clawde/Sonnet, “Targeted Remediation Verification (Findings #1–#4),” archived EASTER research artifact, 1 Oct. 2026, pinned at artifact commit `ef81644f655294ecb7d863a9c6ce527f7fef214f` (merge of PR #16) at `paper/reviews/clawde-draft2-remediation-verification-2026-10-01.md`.

A Comparative Evidence Register

This appendix records the surviving evidentiary basis for the six-system comparative result reported in Section 6. It is a **recovery register**, not a reconstructed original dataset.

The governing rule is:

Missing historical evidence remains missing. This appendix does not infer primitive classifications from aggregate counts, present-day recollection, or later re-review.

The principal recovered artifact is `paper/archive/frozen-review-recovery-2026-09-29.md`, an archival reconstruction created from previously preserved EASTER/DRC research state. That artifact explicitly distinguishes recovered findings from material that was not recovered.

A.1 A.1 Derivation status of the aggregate result

The manuscript reports the frozen per-system aggregate GAP-count sequence, in corpus order:

(0, 0, 0, 0, 1, 3)

The surviving evidence supports that sequence at different resolutions by system. It does **not** support reconstructing a complete primitive-by-system matrix.

This table establishes what can and cannot be derived from the surviving archive. In particular:

- Anthropic’s zero cannot be expanded into six COVERED primitive cells.
- OpenAI’s one cannot be assigned to a primitive.
- Antigravity’s three cannot be reverse-engineered from the six recovered issue families.

A.2 A.2 Recovered system-level evidence

A.2.1 Hermes Agent

Recovered:

System	Frozen aggregate GAP count	Surviving derivation evidence	Irrecoverable / unknown
Hermes Agent	0	Primitive-level EASTER dispositions survive for all six primitives; H-OPEN-1 survives for Transition/Receipt; whole-system freeze records 0 demonstrated GAP.	Exact source commit/version and individual DRC receipts not recovered.
OpenClaw	0	Initial sweep 30 COVERED / 11 OPEN / 0 GAP / 0 CHALLENGED; whole-system primitive dispositions survive; OC-OPEN-1/2 survive; source commit abbreviated as 2ef3b4a0....	Individual DRC primitive receipts not recovered.
LangGraph	0	Substantial Evidence findings; Transition 7 COVERED / 4 OPEN; Exception 7 COVERED / 4 OPEN; Receipt 5 COVERED / 5 OPEN; whole-system freeze records 0 demonstrated GAP.	Exact Authority/State matrices and source SHA/version not recovered.
Anthropic Claude Agent SDK	0	Frozen aggregate 0 GAP and DRC D/R/C COVERED survive.	Primitive-level EASTER classifications, OPEN findings, composition reasoning, and exact reviewed version not recovered.
OpenAI Agents SDK	1	Frozen aggregate records exactly one real GAP; DRC D/R/C COVERED survives.	Owning EASTER primitive, primitive-level COVERED/OPEN findings, and exact reviewed version not recovered. A later recollection about Evidence is excluded.
Google Antigravity	3	Substantial Evidence/Authority/State/Transition/Exception findings survive; six conceptual issue families survive; frozen final aggregate records 3 GAPs; DRC D/R/C COVERED survives.	Receipt freeze, exact mapping from six issue families to final three GAPs not recovered.

- whole-system freeze dated September 24, 2026;
- Evidence COVERED;
- Authority COVERED;
- State COVERED;
- Transition COVERED with H-OPEN-1;
- Exception COVERED;
- Receipt COVERED with H-OPEN-1;
- whole-system 0 demonstrated GAP;
- H-OPEN-1: external effect may occur, local durable record may be lost, no independent external receipt may exist, leaving later recovery unable to establish whether the effect occurred;
- DRC D/R/C COVERED.

Not recovered:

- exact primitive DRC review receipts;
- exact repository commit/version pin.

A.2.2 OpenClaw

Recovered:

- frozen review dated September 24–25, 2026;
- source commit recorded as `2ef3b4a0...`;
- initial primitive sweep: 30 COVERED / 11 OPEN / 0 GAP / 0 CHALLENGED;
- whole-system Evidence COVERED;
- Authority COVERED;
- State COVERED + OC-OPEN-1;
- Transition COVERED + OC-OPEN-1;
- Exception COVERED;
- Receipt COVERED + OC-OPEN-1 + OC-OPEN-2;
- whole-system 0 demonstrated GAP;
- OC-OPEN-1: unrecoverable external-effect window;
- OC-OPEN-2: delivery bypass may leave no authoritative later proof of delivery;

- DRC D/R/C COVERED.

Not recovered:

- individual DRC primitive receipts.

A.2.3 LangGraph

Recovered:

- frozen review dated September 24, 2026;
- whole-system 0 demonstrated GAP;
- Evidence includes COVERED and OPEN findings, including the `UntrackedValue` boundary as OPEN;
- Transition: 7 COVERED / 4 OPEN;
- Exception: 7 COVERED / 4 OPEN;
- Receipt: 5 COVERED / 5 OPEN;
- Authority and State were reviewed/frozen, but exact matrices are not preserved;
- whole-system composition closed multiple primitive-level OPEN findings;
- LANGGRAPH INVARIANT-0: Invariant was considered as a possible seventh primitive and rejected as unnecessary for reconstructing the execution that actually occurred;
- DRC D/R/C COVERED.

Not recovered:

- exact Authority/State finding counts;
- exact source SHA/version pin.

A.2.4 Anthropic Claude Agent SDK

Recovered:

- frozen EASTER aggregate: 0 GAP;
- whole-system disposition: no demonstrated GAP in the frozen comparison;
- DRC D/R/C COVERED.

Not recovered:

- primitive-level EASTER classifications;
- surviving OPEN findings;
- detailed whole-system composition;
- exact reviewed source SHA/version.

No primitive cells are inferred from the zero aggregate.

A.2.5 OpenAI Agents SDK

Recovered:

- frozen EASTER aggregate: exactly 1 GAP;
- existence of a real recorded GAP;
- DRC D/R/C COVERED.

Not recovered:

- which EASTER primitive owned the GAP;
- primitive-level COVERED/OPEN findings;
- exact reviewed source SHA/version.

A later recollection suggested a particular Evidence disposition, but no preserved artifact substantiates it. That recollection is excluded.

A.2.6 Google Antigravity

Recovered primitive material includes:

Evidence

- GAP: MCP annotations lost before policy evaluation;
- GAP: exception fidelity lost;
- COVERED: call/result/step correlation;
- OPEN: truncation behavior;
- OPEN: HookContext / ToolContext fragmentation.

Authority

- GAP families involving retrospective authorization provenance, Boolean human approval, dynamic delegation provenance, and silent ineffective sandbox behavior;

- COVERED: capability/authorization separation and required authorization boundary.

State

- COVERED: recovery, trajectory/workspace separation, concurrency protection;
- GAP: compaction change metadata;
- OPEN: exact effective context and cross-domain provenance.

Transition

- compaction lacks reconstructable transition metadata;
- chained argument transformations lack exposed provenance;
- full finding counts not recovered.

Exception

- GAP: structured exception fidelity;
- COVERED: correlated failures, terminal propagation, appropriate suppression, fail-closed predicates;
- OPEN: predicate diagnostics, retry history, durable diagnostic history.

Receipt

- not sufficiently recovered.

Recovered whole-system issue families:

1. Evidence metadata loss;
2. retrospective authorization provenance;
3. effective-state / compaction reconstruction;
4. tool-call transformation provenance;
5. structured failure fidelity; and
6. normalized terminal outcomes.

Frozen final aggregate: 3 GAPs.

The exact mapping from the six issue families to the final three GAPs was not recovered and is intentionally not reconstructed.

A.3 A.3 Reproducibility boundary

This register makes the surviving derivation inspectable, but it does not make the historical study fully reproducible.

A reader can verify that Section 6 accurately reports the recovered artifact and its explicit unknowns. A reader cannot reproduce every original primitive classification for Anthropic or OpenAI, the exact Antigravity six-to-three composition decision, or several missing source/version pins from the surviving package alone.

Re-running the reverse review against current or pinned system sources would be valuable future work, but it would create a new replication dataset. It would not recover the missing historical evidence and should not be substituted for it.

A.4 A.4 Preservation rule

The evidence register is intentionally asymmetric: some rows are detailed and others are sparse. That asymmetry is itself part of the research record.

The paper therefore prefers an explicit unknown over a retrospectively completed matrix.

B Publication Reference Map

This appendix maps the manuscript’s externally grounded architectural claims to the verified bibliography in §13. It is intentionally separate from the historical-source record: Appendix A and the archived review artifacts describe what the original research pass preserved, while this appendix identifies authoritative publication references a reader can use to inspect the claims now.

B.1 B.1 Comparative corpus navigation

B.2 B.2 Related-work claim map

B.3 B.3 EASTER implementation and evidence map

B.4 B.4 Historical-versus-publication rule

A publication reference answers **where a reader can inspect an authoritative source now**. It does not answer **which exact source/version the historical reviewer inspected** unless the archive independently preserves that fact.

Accordingly:

20 –[25] must not be used to fill unrecovered historical version cells in Appendix A;

3 identifies the same Corda v1.0 whitepaper cited and quoted in the preserved closest-prior evaluation; using the authoritative R3-hosted copy for publication does not change the manuscript’s statement about the historical access path used by Pax/Muse;

Manuscript target	Publication reference	Boundary
Hermes Agent	[20]	Current official documentation for navigation; not an invented historical version pin.
OpenClaw	[21]	Current official session/memory documentation for navigation; historical review evidence remains Appendix A.
LangGraph	[22]	Current official documentation for navigation; historical review evidence remains Appendix A.
Anthropic Claude Agent SDK	[23]	Current official Anthropic agent-runtime documentation for navigation; historical aggregate remains unrecovered below system level.
OpenAI Agents SDK	[24]	Current official SDK documentation for navigation; historical owning primitive for the one-GAP aggregate remains unrecovered.
Google Antigravity	[25]	Current official documentation for navigation; historical six-family-to-three-GAP mapping remains unrecovered.

16 retains RFC 6962 because that specification family belongs to the historical search record even though RFC 9162 supersedes it; and

18 and [19] are explicitly historical architectural references whose current maintenance/service status is disclosed rather than hidden.

This map is intended to make Draft 2 adversarially reviewable before a target venue is selected. Venue conversion may replace bracket numbering/style mechanically, but it must preserve these evidence boundaries.

§9 claim family	Primary / authoritative references
Hyperledger Fabric is a permissioned blockchain architecture with endorsement/validation, immutable transaction history, and a maintained world state	[1], [2]
Corda uses transaction-local state evolution, signatures/participants, attachments, and avoids a single globally broadcast ledger state	[3]
Holochain uses agent source chains, DHT validation, distributed authority, and non-global state; its design explicitly resists externally revocable authorship authority	[4]
W3C PROV provides an interoperable provenance model centered on entities, activities, agents, and their relations	[5], [6]
OAuth 2.0 provides delegated authorization rather than EASTER's continuity semantics	[7]
Capability systems are a mature prior architectural tradition for authority and protected object access	[8], [9]
Temporal represents durable execution through persisted workflow/event history and recovery	[10]
Kafka provides durable partitioned logs / ordered append-oriented record history	[11]
Axon provides event-store/event-sourcing infrastructure and transactional event publication	[12]
EventStoreDB is an event-sourcing database / event-store architecture	[13]
Git provides durable commit history and explicit branching without itself supplying the EASTER admission/outcome conjunction	[14]
CRDT literature provides non-centralized replicated-state convergence mechanisms and is relevant to the non-canonical-state neighborhood	[15]
Certificate Transparency provides append-only auditable logging	[16]
in-toto provides signed supply-chain provenance/attestation and verification	[17]
Ethereum provides durable transaction history/receipts together with protocol state; the Yellow Paper is retained as a historical architecture reference	[18]
Amazon QLDB provided immutable journal/ledger architecture and is retained as historical prior art despite end of service	[19]

Manuscript claim family	Immutable publication evidence
Reference kernel behavior, schema, transaction semantics, Authority behavior, tests	[26]; see <code>PUBLIC-ARTIFACT-MANIFEST.md</code> for pinned entry points.
Accepted remediated Draft 2 baseline and preserved research archive	[27]
Recovered six-system comparative state and explicit unknowns	[28], Appendix A
Pax/Muse prior-art methodology reconstruction	[29]
Clawde/Sonnet prior-art methodology reconstruction	[30]
Reconciliation that produced the four-property behavioral conjunction	[31]
Evidence that the conjunction’s coarse properties predated the dedicated closest-prior investigation	[32]
Clawde/Sonnet’s independent adversarial review of the completed publication package	[33]
Clawde/Sonnet’s targeted verification that review findings #1–#4 were correctly remediated	[34]